

ДИСЦИПЛИНА «Разработка программных средств поиска и анализа киберугроз»

ИНСТИТУТ «Институт кибербезопасности и цифровых технологий»

КАФЕДРА КБ-2 «Информационно-аналитические системы кибербезопасности»

ВИД УЧЕБНОГО МАТЕРИАЛА Лекция

ПРЕПОДАВАТЕЛЬ Новиков Евгений Иванович

СЕМЕСТР 9 семестр 2026-2027 учебный год

ЛЕКЦИЯ 2. СТАТИСТИКА ДЛЯ АНАЛИЗА КИБЕРИНЦИДЕНТОВ: ТИПЫ ПРИЗНАКОВ И ОПИСАНИЕ РАСПРЕДЕЛЕНИЙ

План лекции:

1. Типы признаков.
2. Количественные признаки.
3. Качественные признаки.

Статистика для анализа киберинцидентов: типы признаков и описание распределений

1. Типы признаков

В кибербезопасности природа угроз носит случайный характер, поэтому для их моделирования используются случайные величины.

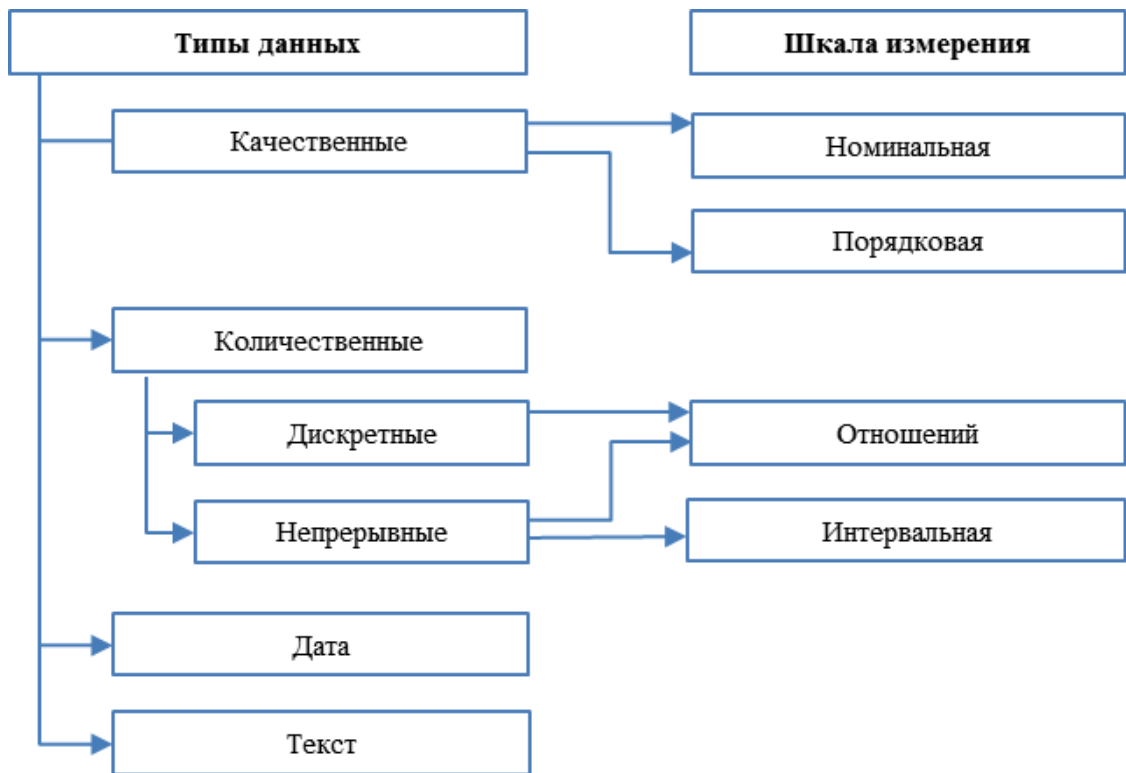
Примером случайной величины может служить задержка реакции SOC (время от срабатывания до ответа), т. е. сколько минут/часов пройдёт от алерта до блокировки угрозы.

Почему это случайная величина:

- загруженность операторов;
- сложность триажа (процесс, который помогает быстро оценивать и приоритизировать оповещения и инциденты на основе риска и срочности. Цель — сосредоточиться на наиболее важных проблемах);
- ложные срабатывания.

Под **случайной величиной** X понимается переменная, которая в результате испытания в зависимости от случая принимает одно из возможных значений: $X = x_1, x_2, \dots, x_n$.

Все признаки (переменные, данные) можно разделить по типу и используемым шкалам измерения.



Качественные (категориальные) признаки - это признаки, которые не могут быть измерены и принимают некоторые имена или метки, и которые можно отнести к определенной группе или номинальной категории на основе некоторого качественного свойства. Обычно они представляют собой строковые значения из ограниченного набора категорий (например, записи в журналах логов, названия вирусов, наличие определенных символов в доменном имени и т. д.). В некоторых случаях могут использоваться и числа, кодирующие эти категории.

Количественные признаки - это признаки, которые могут измерены и принимают числовые значения.

Количественные признаки называются **дискретными**, если множество их значений конечное, или бесконечное, но счётное (значения можно пронумеровать натуральными числами).

Под **непрерывными** количественными признаками понимаются величины, бесконечное несчётное множество значений которых есть некоторый интервал (конечный или бесконечный) числовой оси.

В зависимости от типа и шкалы измерения признаков используются различные технологии их анализа.

Основными средствами анализа данных с использованием языка Python являются библиотеки `pandas`, `numpy`, `matplotlib`, `scipy`, `sklearn`.

2. Количественные признаки

2.1. Описание закона распределения

Наиболее полным описанием признака является закон распределения.

Законом распределения случайной величины называется всякое соотношение, устанавливающее взаимосвязь между возможными значениями случайной величины и соответствующими им вероятностями. Эмпирическим аналогом закона распределения служит ряд распределения (вариационный ряд).

Закон распределения может быть описан следующими способами:

- табличный;
- аналитический;
- графический.

Рассмотрим технологии реализации этих способов с помощью библиотек `Pandas` и `Matplotlib`.

```
In [1]: import warnings
warnings.filterwarnings('ignore')
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
from scipy import stats
%config InlineBackend.figure_format = 'retina'
pd.set_option('display.max_columns', None)
```

Для решения учебных задач будет использоваться набор данных, описывающих сетевой трафик и поведение пользователей.

```
In [2]: # @title
np.random.seed(42)
n_samples = 5000

# === КОЛИЧЕСТВЕННЫЕ ПРИЗНАКИ ===

# 1. ПУАССОНА (дискретное) – частота запросов
req_per_min = np.random.poisson(lam=15, size=n_samples)

# 2. ЛОГНОРМАЛЬНОЕ – размер пейлоада (тяжёлые хвосты)
payload_size = np.random.lognormal(mean=3.0, sigma=1.2, size=n_samples)

# 3. ЭКСПОНЕНЦИАЛЬНОЕ – время отклика
response_time = np.random.exponential(scale=50, size=n_samples)

# 4. НОРМАЛЬНОЕ (Гаусса) – загрузка CPU в штатном режиме
# Среднее 35%, стандартное отклонение 8%
cpu_load = np.random.normal(loc=35, scale=8, size=n_samples)
cpu_load = np.clip(cpu_load, 0, 100) # ограничиваем физическим смыслом

# 5. РАВНОМЕРНОЕ – эфемерный порт источника
source_port = np.random.randint(low=49152, high=65536, size=n_samples)
```

```

# === ЦЕЛЕВАЯ ПЕРЕМЕННАЯ ===
attack_prob = 1 / (1 + np.exp(-(req_per_min/30 + payload_size/500 - 2)))
is_malicious = np.random.binomial(1, attack_prob)

# === КАЧЕСТВЕННЫЕ ПРИЗНАКИ ===
roles = np.random.choice(['admin', 'user', 'guest'], n_samples, p=[0.05, 0.75, 0.20])
methods = np.random.choice(['GET', 'POST', 'PUT', 'DELETE'], n_samples, p=[0.6, 0.2, 0.1, 0.1])

# severity_level
severity_categories = ['Low', 'Medium', 'High', 'Critical']
# Вероятности зависят от вредоносности:
# - Легитимные события: в основном Low/Medium (информационные)
# - Вредоносные события: смещение в сторону High/Critical (требуют реакции)

severity_level = np.empty(n_samples, dtype=object)

# Для легитимного трафика
legit_mask = (is_malicious == 0)
severity_level[legit_mask] = np.random.choice(
    severity_categories,
    size=legit_mask.sum(),
    p=[0.60, 0.30, 0.08, 0.02] # Преимущественно Low/Medium
)

# Для вредоносного трафика
attack_mask = (is_malicious == 1)
severity_level[attack_mask] = np.random.choice(
    severity_categories,
    size=attack_mask.sum(),
    p=[0.05, 0.20, 0.45, 0.30] # Преимущественно High/Critical
)

# === АНОМАЛИИ (выбросы) ===

# Выбираем 50 случайных индексов для жестких аномалий
#anomaly_indices = np.random.choice(n_samples, size=50, replace=False)

# ПРИНУДИТЕЛЬНО помечаем эти строки как вредоносные
#is_malicious[anomaly_indices] = 1

# Инжектируем аномалии ТОЛЬКО в эти строки
#response_time[anomaly_indices] *= 10 # Резкий рост задержки (в 10 раз)
#cpu_load[anomaly_indices] = np.random.uniform(85, 99, size=50) # Выход за пределы 3-

# Инжектируем сканирование портов (берем первые 30 индексов из аномальных)
#port_scan_indices = anomaly_indices[:30]
#source_port[port_scan_indices] = np.arange(49152, 49152 + 30)

# === СОХРАНЕНИЕ ===
df = pd.DataFrame({
    'user_role': roles,
    'request_method': methods,
    'requests_per_min': req_per_min,
    'payload_size_bytes': payload_size,
    'response_time_ms': response_time,
    'cpu_load_percent': cpu_load,
    'source_port': source_port,
    'severity_level': severity_level,
    'is_malicious': is_malicious
})
df.head()

```

Out[2]:

	user_role	request_method	requests_per_min	payload_size_bytes	response_time_ms	cpu_lc
0	user	PUT	18	36.412739	98.430919	
1	user	GET	10	43.521982	7.556738	
2	guest	POST	16	8.897820	28.100661	
3	user	POST	19	13.921030	5.114064	
4	user	GET	11	9.807454	24.681651	



Признаки:

- user_role: тип пользователя;
- request_method: тип запроса;
- requests_per_min: частота запросов;
- payload_size_bytes: размер пейлоада;
- response_time_ms: время отклика;
- cpu_load_percent: загрузка CPU;
- source_port: порт источника;
- is_malicious: является вредоносным

Дискретные признаки

Табличный способ

Табличный способ описания закона распределения дискретных признаков называется **рядом распределения** и представляется в виде двух строк (столбцов). В первой строке записываются все возможные значения признака в порядке возрастания, во второй - вероятности их появления.

Значения СВ	x_1	x_2	...	x_i	...	x_n
генеральная совокупность	p_1	p_2	...	p_i	...	p_n
выборочная совокупность	w_1	w_2	...	w_i	...	w_n

Определим дискретные признаки и количество их уникальных значений.

```
In [3]: df.select_dtypes(include = 'int64').nunique()
```

Out[3]:

	0
requests_per_min	27
source_port	4263
is_malicious	2

dtype: int64

Для построения ряда распределения в библиотеке `pandas` можно использовать функцию `value_counts()`.

Построим ряд распределения признака `requests_per_min` - частота запросов.

```
In [4]: # Абсолютные частоты.
freqa = df['requests_per_min'].value_counts().values

# Относительные частоты.
freqo = df['requests_per_min'].value_counts(normalize = True).values

# Создание таблицы и добавление в нее компонентов закона распределения.
df_d_distr = pd.DataFrame()
df_d_distr['requests_per_min'] = df['requests_per_min'].value_counts().index
df_d_distr['Абсолютная_частота'] = freqa
df_d_distr['Относительная_частота'] = freqo
df_d_distr = df_d_distr.sort_values(by = 'requests_per_min')
df_d_distr
```

Out[4]:

	requests_per_min	Абсолютная_частота	Относительная_частота
26	3	1	0.0002
23	4	5	0.0010
21	5	15	0.0030
19	6	26	0.0052
16	7	46	0.0092
14	8	98	0.0196
11	9	153	0.0306
9	10	247	0.0494
7	11	341	0.0682
4	12	444	0.0888
3	13	465	0.0930
0	14	511	0.1022
1	15	490	0.0980
2	16	485	0.0970
5	17	436	0.0872
6	18	344	0.0688
8	19	281	0.0562
10	20	191	0.0382
12	21	134	0.0268
13	22	102	0.0204
15	23	85	0.0170
17	24	42	0.0084
18	25	29	0.0058
20	26	16	0.0032
22	27	8	0.0016
25	28	2	0.0004
24	29	3	0.0006

Аналитический способ

Для аналитического способа описания дискретных признаков используются следующие функции:

- функция вероятностей (функция массы вероятности – Probability Mass Function - PMF);
- функция распределения вероятностей (кумулятивная функция распределения – Cumulative distribution function - CDF).

1. Функция вероятностей (PMF) – это функция, которая дает вероятность того, что дискретная случайная величина в точности равна некоторому значению:

$$p(x) = p(X = x).$$

Если мы анализируем выборочные данные, то можно говорить только об **эмпирической** функции вероятностей (EPMF), в которой неизвестные вероятности заменяются их оценками – относительными частотами:

$$\tilde{p}(x) = w(X = x).$$

Эмпирическую функцию вероятностей можно записать, используя данные ряда распределения.

Для признака `requests_per_min` фрагмент эмпирической функции вероятностей будет иметь следующий вид:

$$\tilde{p}(x) = \begin{cases} 0.0002, & \text{если } x = 3 \\ 0.0010, & \text{если } x = 4 \\ \dots\dots\dots \\ 0.0004, & \text{если } x = 28 \\ 0.0006, & \text{если } x = 29 \end{cases} \quad (1)$$

2. Функция распределения вероятностей (CDF) случайной величины X называется функция $F(x)$, выражающая вероятность того, что случайная величина X примет значение, меньшее некоторого заданного x : $F(x) = p(X < x)$. Эту функцию также называют **интегральной** функцией распределения.

В дискретном случае событие $X < x$ состоит из всех элементарных событий $X < x_i$, таких, что $x_i < x$. Поэтому в соответствии с определением вероятности составного события можно записать:

$$F(x) = p(X < x) = \sum_{i: x_i < x} p(X = x_i) = \sum_{i: x_i < x} p_i.$$

Эмпирическая функция распределения вероятностей (ECDF):

$$\tilde{F}(x) = \tilde{p}(X < x) = \sum_{i: x_i < x} w(X = x_i) = \sum_{i: x_i < x} w_i.$$

Для формирования функции распределения предварительно необходимо вычислить накопленные частоты. С этой целью можно использовать метод `cumsum()`.

Примечания:

1. Метод `cumsum()` вычисляет накопленные частоты исходя из нестрогого неравенства $F(x) = p(X \leq x)$. Чтобы это исправить нужно сдвинуть значения функции распределения на 1 строку вниз, дополнив первую строку 0.
2. Для полноты описания закона распределения накопленная относительная частота, равная 1 должна соответствовать событию $x_{max} + 1 = 29 + 1 = 30$:

$$\tilde{F}(31) = \tilde{p}(X < 30) = \sum_{i: x_i < 30} \tilde{p}(X = x_i) = \sum_{i: x_i < 30} w_i.$$

```
In [5]: # Создание таблицы со значениями признака и частотами
cdf_table = pd.DataFrame({'requests_per_min': df_d_distr['requests_per_min'],
                          'Абсолютная_частота': df_d_distr['Абсолютная_частота'],
                          'P(X=x)': df_d_distr['Относительная_частота'].round(4)})

# Расчет функции распределения  $F(x) = P(X < x)$ 
# cumsum() даёт  $P(X \leq x)$ , сдвигаем на 1 строку вниз → получаем  $P(X < x)$ 
# fillna(0) заполняет первую строку нулём (для  $x_{\min}$ )
cdf_table['F(x) = P(X < x)'] = cdf_table['P(X=x)'].cumsum().shift(1).fillna(0).round(4)

# Добавление последней строки:  $F(x_{\max} + \varepsilon) = 1$ 
# Это важно для полноты описания закона распределения
last_x = cdf_table['requests_per_min'].iloc[-1]
final_row = pd.DataFrame({'requests_per_min': last_x + 1,
                          'Абсолютная_частота': 0,
                          'P(X=x)': 0.0,
                          'F(x) = P(X < x)': 1.0})
cdf_table = pd.concat([cdf_table, final_row], ignore_index=True)
cdf_table
```

Out[5]:

	requests_per_min	Абсолютная_частота	P(X=x)	F(x) = P(X < x)
0	3	1	0.0002	0.0000
1	4	5	0.0010	0.0002
2	5	15	0.0030	0.0012
3	6	26	0.0052	0.0042
4	7	46	0.0092	0.0094
5	8	98	0.0196	0.0186
6	9	153	0.0306	0.0382
7	10	247	0.0494	0.0688
8	11	341	0.0682	0.1182
9	12	444	0.0888	0.1864
10	13	465	0.0930	0.2752
11	14	511	0.1022	0.3682
12	15	490	0.0980	0.4704
13	16	485	0.0970	0.5684
14	17	436	0.0872	0.6654
15	18	344	0.0688	0.7526
16	19	281	0.0562	0.8214
17	20	191	0.0382	0.8776
18	21	134	0.0268	0.9158
19	22	102	0.0204	0.9426
20	23	85	0.0170	0.9630
21	24	42	0.0084	0.9800
22	25	29	0.0058	0.9884
23	26	16	0.0032	0.9942
24	27	8	0.0016	0.9974
25	28	2	0.0004	0.9990
26	29	3	0.0006	0.9994
27	30	0	0.0000	1.0000

Аналитический вид эмпирической функции распределения (ECDF):

$$\hat{F}(x) = \begin{cases} 0, & \text{если } x \leq 3 \\ 0.0002, & \text{если } 3 < x \leq 4 \\ 0.0012, & \text{если } 4 < x \leq 5 \\ \dots\dots\dots & \\ 1, & \text{если } x \leq 30 \end{cases} \quad (2)$$

Графический способ

Графический способ описания закона распределения дискретных признаков заключается в построении графиков эмпирических функций вероятностей (EPMF) и распределения вероятностей (ECDF).

Для графического представления функции вероятностей используется специальный график – **полигон распределения** или **многоугольник распределения**. На этом графике по оси абсцисс откладываются значения признака, а по оси ординат – соответствующие им вероятности (относительные частоты).

```
In [6]: plt.figure(figsize = (16, 14))
df_d_distr.plot('requests_per_min',
                'Относительная_частота',
                marker = 'o',
                legend = False)
plt.title('Полигон частот')
plt.xlabel('requests_per_min')
plt.ylabel('Относительная_частота')
plt.grid();
```

<Figure size 1600x1400 with 0 Axes>

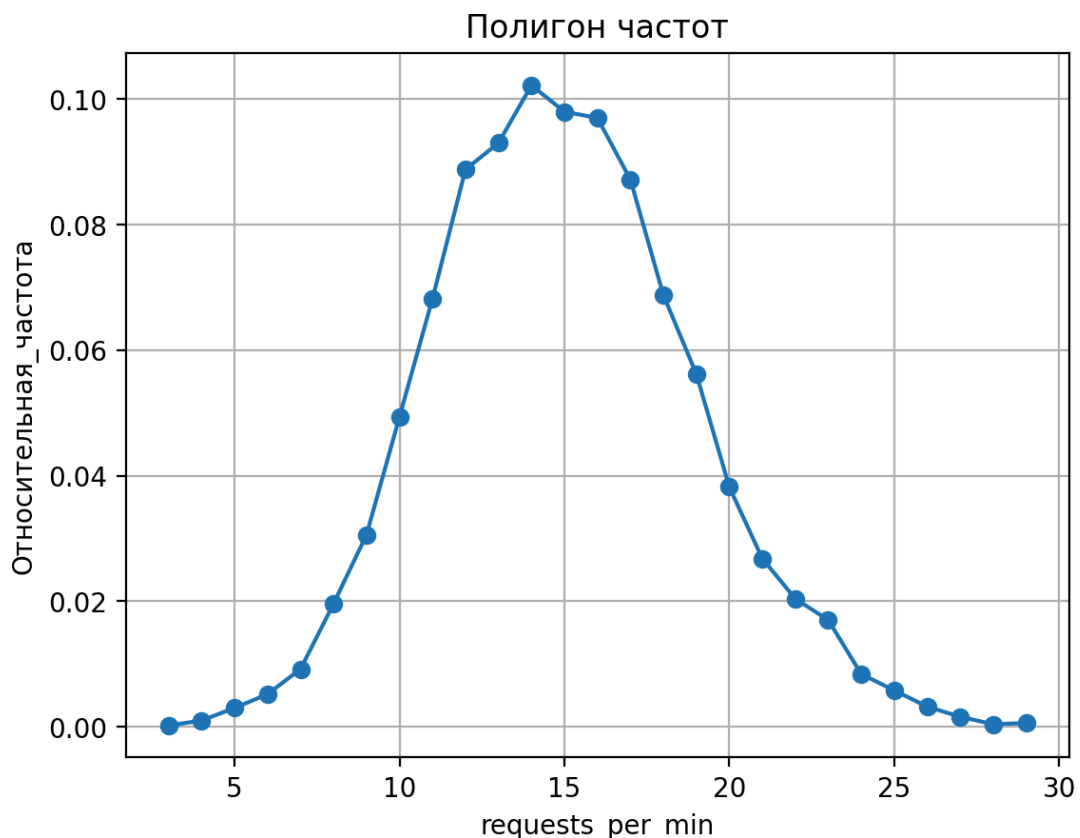


График эмпирической функции распределения называется **кумулятой**. Для дискретной случайной величины этот график имеет ступенчатый вид, ее скачки соответствуют значениям признака и равны накопленным вероятностям (относительным частотам).

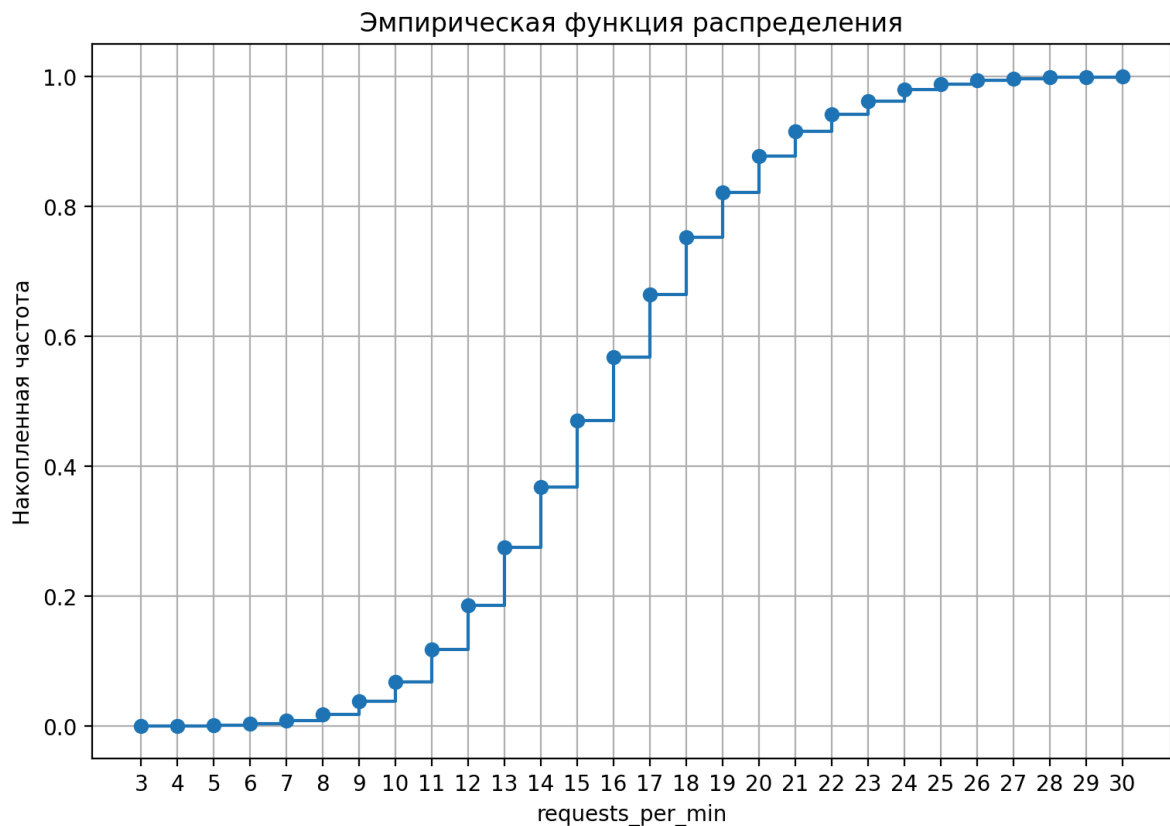
Для построения графика используется функция `step`. Чтобы выбрать конфигурацию графика используется атрибут `where = {'pre', 'post', 'mid'}`.

```
In [7]: plt.figure(figsize = (9, 6))
plt.step(cdf_table['requests_per_min'],
```

```

cdf_table['F(x) = P(X < x)'],
where = 'post', marker = 'o')
plt.title('Эмпирическая функция распределения')
plt.ylabel('Накопленная частота')
plt.xlabel('requests_per_min')
plt.xticks(np.arange(3, 31))
plt.grid()

```



Непрерывные признаки

Табличный способ

Вероятность того, что непрерывный признак примет конкретное значение равна нулю. Поэтому записать закон распределения в виде таблицы по аналогии с дискретными признаками невозможно.

Вместо вероятности конкретного значения оценивается вероятность попадания признака в заданные интервалы. Тогда можно записать закон распределения непрерывного признака в виде таблицы.

Интервал	$[x_1; x_2)$	$[x_2; x_3)$	$[x_3; x_4)$		$[x_m; x_{m+1})$
генеральная совокупность	p_1	p_2	p_3		p_m
выборочная совокупность	w_1	w_2	w_3	...	w_m

Процедура формирования интервалов получила название биннинг (binning).

Основные этапы:

- определение количества интервалов;

- определение ширины интервалов;
- расчет частот попадания значений признака в каждый интервал.

Для формирования интервалов рекомендуется использовать функцию `cut()` :

- `x` - входной массив, подлежащий бинированию (должен быть одномерным);
- `bins` - количество интервалов или список границ интервалов:
 - `int` - определяет количество одинаковых по ширине интервалов (диапазон расширяется на 0.1% с каждой стороны, чтобы включить минимальное и максимальное значения признака);
 - последовательность скаляров - определяет границы бинов, допускающие неравномерную ширину;
 - `IntervalIndex` - определяет точные бины, которые будут использоваться;
- `right` (по умолчанию `True`) - указывает, включают ли бины крайнее правое значение или нет, если `right = True`, то бины `[1, 2, 3, 4]` обозначают `(1,2]`, `(2,3]`, `(3,4]` (этот аргумент игнорируется, если `bins` - это `IntervalIndex`);
- `labels` (по умолчанию `None`) - задает метки для возвращаемых бинов, они должны быть той же длины, что и результирующие бины, если `False`, то возвращаются только целочисленные показатели бинов (этот аргумент игнорируется, если `bins` - это `IntervalIndex`);
- `retbins` (по умолчанию `False`) - возвращать или нет бины;
- `precision` (по умолчанию 3) - точность, с которой следует хранить и отображать метки бинов;
- `include_lowest` (по умолчанию `False`) - включать или не включать первый интервал в левую часть;
- `ordered` (по умолчанию `True`) - упорядочены ли метки или нет.

При построении ряда распределения непрерывного признака ключевым моментом является выбор количества и ширины интервалов.

Количество интервалов определяется по правилу Стёрджеса:

$$k = 1 + 3.322 \cdot \log_{10}(n),$$

где n - количество наблюдений в наборе данных.

Ширину интервалов можно определить по правилу Скотта:

$$h = 3.5 \cdot \frac{\sigma}{n^{1/3}}.$$

Определим непрерывные признаки.

```
In [8]: df.select_dtypes(include = 'float64').head(2)
```

```
Out[8]:
```

	payload_size_bytes	response_time_ms	cpu_load_percent
0	36.412739	98.430919	40.553688
1	43.521982	7.556738	23.411752

Рассмотрим пример построения ряда распределения признака `cpu_load_percent` - загрузки CPU.

In [9]: *# ШАГ 1: Определяем количество интервалов*

```
n = len(df)
k = int(1 + 3.322 * np.log10(n))
print(f'Количество интервалов: {k}')
```

Количество интервалов: 13

In [10]: *# ШАГ 2: Определяем ширину интервалов*

```
# Для cpu_load_percent (0-100%) логично использовать
# равные интервалы шириной 5%
bins = np.arange(0, 105, 5)
bins
```

Out[10]: array([0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100])

In [11]: *# ШАГ 3: Разбиваем данные на интервалы с помощью pd.cut()*

```
# pd.cut() ставит в соответствие каждому наблюдению
# соответствующий интервал
# right = False означает, что интервалы [a, b), а не (a, b]
df['cpu_load_bin'] = pd.cut(df['cpu_load_percent'],
                           bins = bins,
                           right = False)
df['cpu_load_bin']
```

Out[11]:

	cpu_load_bin
0	[40, 45)
1	[20, 25)
2	[20, 25)
3	[25, 30)
4	[30, 35)
...	...
4995	[40, 45)
4996	[20, 25)
4997	[50, 55)
4998	[35, 40)
4999	[30, 35)

5000 rows × 1 columns

dtype: category

In [12]: *# ШАГ 4: Строим ряд распределения*

```
# value_counts() считает частоты, sort = False сохраняет порядок интервалов
```

```
freq_series = df['cpu_load_bin'].value_counts(sort = False).sort_index()
freq_series
```

Out[12]:

cpu_load_bin	count
[0, 5)	1
[5, 10)	5
[10, 15)	24
[15, 20)	115
[20, 25)	385
[25, 30)	825
[30, 35)	1158
[35, 40)	1174
[40, 45)	779
[45, 50)	385
[50, 55)	131
[55, 60)	15
[60, 65)	3
[65, 70)	0
[70, 75)	0
[75, 80)	0
[80, 85)	0
[85, 90)	0
[90, 95)	0
[95, 100)	0

dtype: int64

```
In [13]: # Создаём таблицу ряда распределения
dist_table = pd.DataFrame({'Интервал': freq_series.index,
                           'Абсолютная_частота': freq_series.values,
                           'Относительная_частота': (freq_series.values / n).round(4)})
dist_table
```

Out[13]:

	Интервал	Абсолютная_частота	Относительная_ частота
0	[0, 5)	1	0.0002
1	[5, 10)	5	0.0010
2	[10, 15)	24	0.0048
3	[15, 20)	115	0.0230
4	[20, 25)	385	0.0770
5	[25, 30)	825	0.1650
6	[30, 35)	1158	0.2316
7	[35, 40)	1174	0.2348
8	[40, 45)	779	0.1558
9	[45, 50)	385	0.0770
10	[50, 55)	131	0.0262
11	[55, 60)	15	0.0030
12	[60, 65)	3	0.0006
13	[65, 70)	0	0.0000
14	[70, 75)	0	0.0000
15	[75, 80)	0	0.0000
16	[80, 85)	0	0.0000
17	[85, 90)	0	0.0000
18	[90, 95)	0	0.0000
19	[95, 100)	0	0.0000

Аналитический способ

Для аналитического способа описания непрерывных признаков используются следующие функции:

- функция плотности распределения вероятностей (probability density function - PDF) - аналог функции вероятностей (PMF) для дискретных признаков;
- функция распределения вероятностей (кумулятивная функция распределения – Cumulative distribution function - CDF).

Функция плотности распределения вероятностей (PDF) $f(x)$ – есть вероятность, которая приходится на единицу длины, рассматриваемого участка и равна производной интегральной функции распределения $F(x)$. Поэтому $f(x)$ называют **дифференциальной** функцией распределения. Смысл плотности распределения $f(x)$ состоит в том, что она указывает на то, как часто появляется случайная величина X в некоторой окрестности точки x при повторении опытов.

По выборочным данным можно построить только эмпирическую функцию плотности распределения. Чтобы записать ее аналитическое выражение производится анализ (по

таблицу или графику) на предмет схожести с известными теоретическими функциями плотности распределения.

Функция распределения вероятностей непрерывной случайной величины X называется функция $F(X)$, выражающая вероятность того, что случайная величина X примет значение, меньшее x :

$$F(x) = p(X < x).$$

Функция распределения непрерывной случайной величины имеет тот же смысл, что и в случае дискретной величины, но только является непрерывной функцией в генеральной совокупности.

По выборочным данным можно построить только эмпирическую функцию распределения. Чтобы записать ее аналитическое выражение достаточно проинтегрировать функцию плотности распределения вероятностей:

$$F(x) = \int_{-\infty}^x f(x)dx.$$

Графический способ

Графический способ описания закона распределения непрерывных признаков заключается в построении графиков эмпирических функций плотности распределения вероятностей (EPDF) и распределения вероятностей (ECDF).

Для построения графика функции распределения непрерывного признака предварительно необходимо вычислить ее значения. С этой целью можно использовать пакет `statsmodels`.

`Statsmodels` - это пакет Python, который позволяет проводить статистическое оценивание числовых характеристик случайных величин, выполнять статистические тесты и оценивать статистические модели. Для разных типов данных и каждого оценщика доступен обширный список описательной статистики, статистических тестов, функций построения графиков и статистики результатов.

`Statsmodels` построен на основе числовых библиотек `NumPy` и `SciPy`, интегрируется с `Pandas` для обработки данных. Графические функции основаны на библиотеке `Matplotlib`.

Для расчета эмпирической функции распределения используется функция `ECDF()` модуля `distributions.empirical_distribution`.

```
In [14]: from statsmodels.distributions.empirical_distribution import ECDF
# расчет эмпирической функции распределения
ecdf = ECDF(df['cpu_load_percent'])
# исходные значения признака (x)
# и вычисленные накопленные вероятности (y)
ecdf.x, ecdf.y
```

```
Out[14]: (array([      -inf,  4.14899737,  5.75930788, ..., 60.09348839,
        63.82265709, 63.8908005 ]),
        array([0.000e+00, 2.000e-04, 4.000e-04, ..., 9.996e-01, 9.998e-01,
        1.000e+00]))
```

Полученные значения признака и соответствующие накопленные вероятности подаются на вход функций `plot()` или `step()`.

После применения `step()` график будет иметь ступенчатый вид, но поскольку, как правило, непрерывная случайная величина имеет достаточно большое количество не повторяющихся значений, то график будет близок к непрерывной форме.

```
In [15]: plt.figure(figsize = (6, 4))
plt.step(ecdf.x, ecdf.y)
plt.title('Эмпирическая функция распределения')
plt.ylabel('Вероятность')
plt.xlabel('cpu_load_percent')
plt.grid()
```

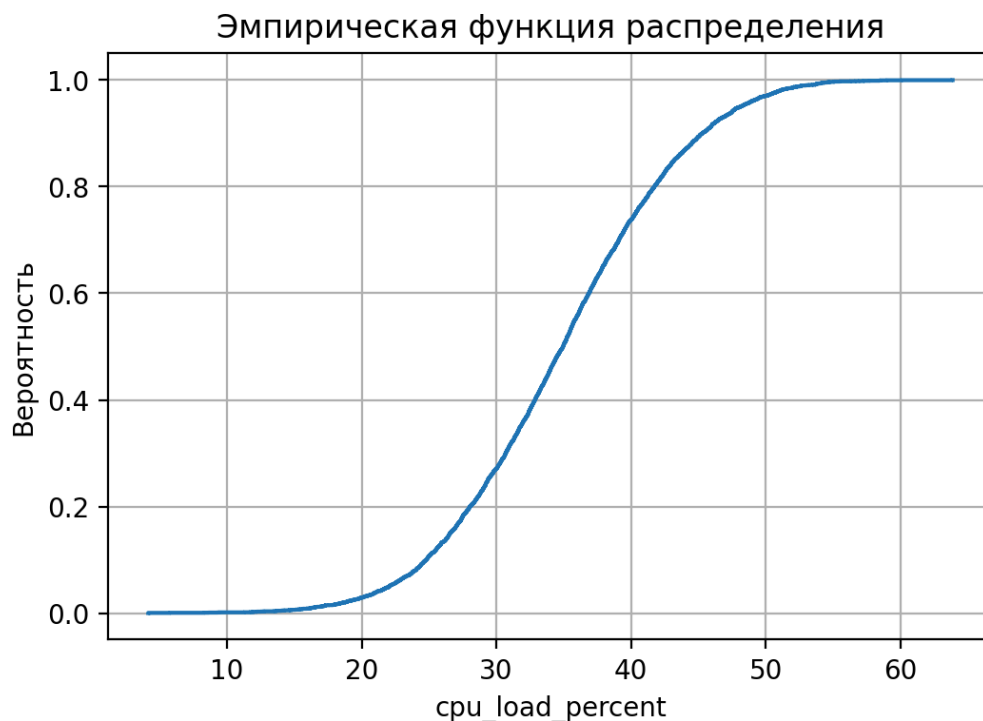


График функции плотности распределения вероятностей (PDF) получил специальное название - гистограмма.

Гистограмма - ступенчатая фигура из прямоугольников с основаниями, равными интервалам значения признака $[x_i, x_{i+1}]$, и высотами, равными плотности вероятности $\tilde{f}(x)$, приходящимся на единицу ширины интервала с номером i :

$$\tilde{f}(x) = n_i / (n \cdot h_i) = w_i / h_i,$$

где n_i – количество значений выборочной случайной величины, которые попали в i -й интервал группирования, h_i – ширина интервала.

На практике часто в качестве высоты прямоугольников используются абсолютные, либо относительные частоты попадания в соответствующие интервалы:

$$\tilde{f}(x) = n_i.$$

$$\tilde{f}(x) = n_i/n = w_i.$$

В случае равных интервалов они имеют одинаковый вид и отличаются только масштабом. Если используются интервалы различной ширины, то необходимо строить классическую гистограмму.

Для построения гистограммы средствами `pandas` можно использовать функцию `hist()`.

Атрибут `bins` задает количество интервалов, атрибут `density` - вид гистограммы, если `True` строится классическая гистограмма, в противном случае - гистограмма абсолютных частот.

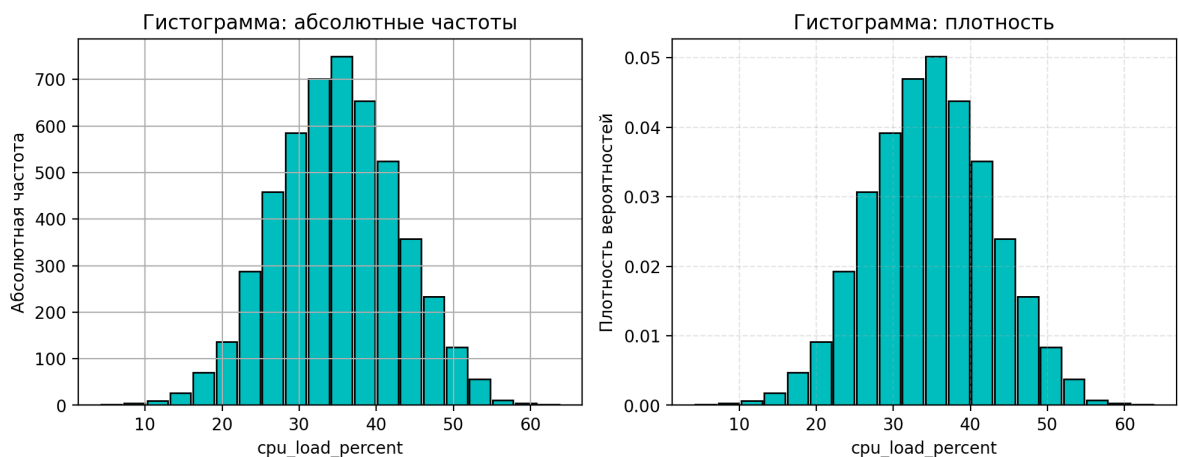
В качестве аргумента параметра `bins` можно использовать результаты построения интервалов с помощью функции `cut()`, либо непосредственно задать необходимое количество.

```
In [16]: fig, axs = plt.subplots(1, 2, figsize=(10, 4))

# Гистограмма абсолютных частот
axs[0].hist(df['cpu_load_percent'], bins=20, color='c',
            edgcolor='black', rwidth=0.9)
axs[0].set_title('Гистограмма: абсолютные частоты')
axs[0].set_xlabel('cpu_load_percent')
axs[0].set_ylabel('Абсолютная частота')
axs[0].grid()

# Гистограмма плотностей
axs[1].hist(df['cpu_load_percent'], bins=20, density=True, color='c',
            edgcolor='black', rwidth=0.9)
axs[1].set_title('Гистограмма: плотность')
axs[1].set_xlabel('cpu_load_percent')
axs[1].set_ylabel('Плотность вероятностей')
axs[1].grid(True, linestyle='--', alpha=0.3)

plt.tight_layout()
```



2.2. Оценивание числовых характеристик

Числовые характеристики случайной величины - это числа, которые описывают случайную величину суммарно. Также их часто называют статистиками.

Минимальное и максимальное значения признака

```
In [17]: print(df['cpu_load_percent'].max())
print(df['cpu_load_percent'].min())
```

```
63.89080050252535
4.148997366075225
```

Математическое ожидание

Дискретный признак: $M(x) = \sum_{i=1}^N x_i \cdot p_i$,

N - объем генеральной совокупности.

Непрерывный признак: $M(x) = \int_{-\infty}^{+\infty} x \cdot f(x) dx$.

Оценка математического ожидания - выборочное среднее арифметическое:

$$\hat{M}(x) = \bar{x} = \sum_{i=1}^m \frac{x_i \cdot n_i}{n},$$

- n - объем выборочной совокупности,
- m - число уникальных значений дискретного признака, либо число интервалов непрерывного признака,
- n_i - частота встречаемости i -го уникального значения дискретного признака, либо количество значений непрерывного признака, попавших в i -й интервал.

```
In [18]: md = df['requests_per_min'].mean()
mn = df['cpu_load_percent'].mean()
print(f'Среднее значение requests_per_min = {md}')
print(f'Среднее значение cpu_load_percent = {mn}')
```

```
Среднее значение requests_per_min = 14.9758
Среднее значение cpu_load_percent = 34.92446556910189
```

Медиана

Медиана (Me) дискретной величины — это значение, которое делит упорядоченный вариационный ряд на две равные части: 50% значений меньше медианы, 50% — больше.

Если число наблюдений n нечетное: $Me = x_{n+1}/2$.

Если число наблюдений n четное: $Me = \frac{x_{n/2} + x_{n/2+1}}{2}$.

```
In [19]: me = df['requests_per_min'].median()
print(f'Медианное значение cpu_load_percent = {me}')
```

```
Медианное значение cpu_load_percent = 15.0
```

Интерпретация в кибербезопасности.

Если медиана признака `requests_per_min` (частота запросов) равна 15, это значит, что в 50% случаев легитимный пользователь генерирует 15 или меньше запросов в минуту. Это надежный базовый уровень (baseline) для настройки правил Web Application Firewall (WAF).

Для непрерывного признака точное значение медианы находится через медианный интервал.

Медианный интервал — это первый интервал, накопленная частота которого превышает половину общего числа наблюдений ($n/2$ или 0.5 для относительных частот).

Формула линейной интерполяции для медианы:

$$Me = x_0 + h \cdot \frac{0.5 \cdot n - S_{Me-1}}{f_{Me}},$$

где:

- x_0 — нижняя граница медианного интервала;
- h — ширина интервала;
- n — общее число наблюдений (сумма частот);
- S_{Me-1} — накопленная частота интервала, предшествующего медианному.
- f_{Me} — частота самого медианного интервала.

```
In [20]: # 1. Используем таблицу интервалов, которую строили ранее
# (предполагаем, что bins = np.arange(0, 105, 5))

freq_series = df['cpu_load_bin'].value_counts(sort=False).sort_index()
median_table = pd.DataFrame({'interval': freq_series.index,
                             'freq': freq_series.values})

# 2. Добавляем колонку накопленной частоты
median_table['cum_freq'] = median_table['freq'].cumsum()

# 3. Находим медианный интервал (где накопленная частота впервые >= N / 2)
N = len(df)
target_freq = N / 2

# Находим индекс первой строки, где cum_freq >= target_freq
median_idx = median_table[median_table['cum_freq'] >= target_freq].index[0]
median_row = median_table.loc[median_idx]

# 4. Извлекаем параметры для формулы
x0 = median_row['interval'].left
h = median_row['interval'].length
f_me = median_row['freq']

# Накопленная частота ДО медианного интервала
s_me_prev = median_table.loc[median_idx - 1, 'cum_freq'] if median_idx > 0 else 0

# 5. Вычисляем медиану по формуле
me_continuous = x0 + h * ((target_freq - s_me_prev) / f_me)

print(f"--- Расчет медианы для непрерывного признака ---")
print()
print(f"Медианный интервал: {median_row['interval']}")
print(f"Параметры: x0={x0}, h={h}, f_me={f_me}, S_Me-1={s_me_prev}, n/2={target_freq}")
print(f"Расчетная медиана (Me): {me_continuous:.2f}%")
```

--- Расчет медианы для непрерывного признака ---

Медианный интервал: [30, 35)

Параметры: $x_0=30$, $h=5$, $f_{me}=1158$, $S_{Me-1}=1355$, $n/2=2500.0$

Расчетная медиана (Me): 34.94%

```
In [21]: # Проверка встроенным методом pandas
print(f"Медиана (встроенный метод pandas): {df['cpu_load_percent'].median():.2f}%")
```

Медиана (встроенный метод pandas): 34.96%

Мода

Мода M_o дискретного признака – это его наиболее вероятное значение.

Выборка может быть унимодальной (одна мода), бимодальной (две моды) или мультимодальной.

```
In [22]: mode_discrete = df['requests_per_min'].mode()
print(f"Мода признака requests_per_min = {mode_discrete.values[0]}")
```

Мода признака requests_per_min = 14

Интерпретация в кибербезопасности.

Если модой является 14 запросов в минуту, это означает, что наиболее типичное (базовое) поведение легитимного пользователя или узла сети — это 14 запросов. Любое значительное отклонение от моды (например, 150 запросов) является кандидатом на проверку (аномалия).

Мода M_o непрерывного признака определяется как такое его возможное значение, при котором значение функции плотности распределения вероятностей $f(x)$ имеет максимум.

Для непрерывного признака точные значения (например, 35.123456) почти никогда не повторяются. Поэтому прямое применение `mode()` бессмысленно:

- будут возвращены все уникальные значения;
- будет найдено значение, которое случайно повторилось чаще других. Это статистический шум, а не истинная мода распределения.

```
In [23]: df['cpu_load_percent'].mode()
```

Out[23]:

cpu_load_percent	
0	4.148997
1	5.759308
2	8.320858
3	8.635511
4	9.296371
...	...
4995	58.542021
4996	58.989873
4997	60.093488
4998	63.822657
4999	63.890801

5000 rows × 1 columns

dtype: float64

Для вычисления моды всегда нужно использовать интервальную группировку для непрерывных величин.

Алгоритм нахождения моды для непрерывных признаков состоит из двух шагов:

1. Найти модальный интервал — интервал группировки с наибольшей частотой.
2. Вычислить точное значение моды внутри этого интервала с помощью формулы линейной интерполяции:

$$M_o = x_0 + h \cdot \frac{f_{M_0} - f_{M_{0-1}}}{(f_{M_0} - f_{M_{0-1}}) + (f_{M_0} - f_{M_{0+1}})},$$

где:

- x_0 — нижняя граница модального интервала;
- h — ширина модального интервала;
- f_{M_0} — частота модального интервала;
- $f_{M_{0-1}}$ — частота интервала, предшествующего модальному;
- $f_{M_{0+1}}$ — частота интервала, следующего за модальным.

In [24]:

```
# Шаг 1: Создаем интервалы
bins = np.arange(0, 105, 5) # Интервалы по 5%: [0, 5), [5, 10), ..., [100, 105)
df['cpu_load_bin'] = pd.cut(df['cpu_load_percent'], bins=bins, right=False)

# Считаем частоты и сортируем по интервалам
freq_series = df['cpu_load_bin'].value_counts(sort=False).sort_index()

# Создаем таблицу для удобства извлечения данных
modal_table = pd.DataFrame({'interval': freq_series.index,
                           'freq': freq_series.values})

# Шаг 2: Находим модальный интервал (где частота максимальна)
```

```

modal_idx = modal_table['freq'].idxmax()
modal_row = modal_table.loc[modal_idx]

# Извлекаем параметры для формулы
x0 = modal_row['interval'].left           # Нижняя граница
h = modal_row['interval'].length          # Ширина интервала
f_mo = modal_row['freq']                  # Частота модального интервала

# Частота предыдущего интервала (если модальный первый, берем 0)
f_mo_prev = modal_table.loc[modal_idx - 1, 'freq'] if modal_idx > 0 else 0

# Частота следующего интервала (если модальный последний, берем 0)
f_mo_next = modal_table.loc[modal_idx + 1, 'freq'] if modal_idx < len(modal_table) - 1 else 0

# Шаг 3: Вычисляем моду по формуле интерполяции
numerator = f_mo - f_mo_prev
denominator = (f_mo - f_mo_prev) + (f_mo - f_mo_next)

mode_continuous = x0 + h * (numerator / denominator)

print(f"--- Расчет моды для непрерывного признака ---")
print()
print(f"Модальный интервал: {modal_row['interval']}")
print(f"Частоты: пред.= {f_mo_prev}, мод.= {f_mo}, след.= {f_mo_next}")
print(f"Расчетная мода (Mo): {mode_continuous:.2f}%")

```

--- Расчет моды для непрерывного признака ---

Модальный интервал: [35, 40)
Частоты: пред.= 1158, мод.= 1174, след.= 779
Расчетная мода (Mo): 35.19%

Можно использовать более простой и понятный способ вычисления моды - через гистограмму. Идея: разбить данные на бины, найти бин с наибольшей частотой, взять его середину как оценку моды.

```

In [25]: X = df['cpu_load_percent'].values
counts, bins = np.histogram(X, bins=20)
max_idx = np.argmax(counts)
mode_est = (bins[max_idx] + bins[max_idx+1]) / 2
print("Оценка моды (по гистограмме):", mode_est)

```

Оценка моды (по гистограмме): 35.51344401271154

2.3. Оценивание вида закона распределения количественных признаков

В кибербезопасности понимание законов распределения — это основа для построения систем обнаружения вторжений (IDS/IPS), настройки порогов срабатывания (thresholds) и выбора правильных алгоритмов машинного обучения.

При построении таких систем «изучается» нормальная, легитимная активность: какие значения типичны для разных метрик (время отклика, размер пакета, частота запросов, длительность сессии и т.п.), какое у этих значений распределение (нормальное, экспоненциальное и др.). На основе этого строится модель «нормы». А когда в потоке данных появляется значение, которое сильно выбивается из установленного паттерна (например, время отклика резко выросло и попало в «тяжёлый хвост» распределения), система помечает это как потенциальную аномалию — возможный признак атаки.

Почему это важно на практике.

- **Снижение ложных срабатываний.** Если модель не учитывает реальную форму распределения, она может ошибочно считать редкое, но легитимное событие (например, разовый тяжёлый запрос от клиента) угрозой. Понимание распределения помогает точнее настроить пороги: не отрезать «хвосты» слишком жёстко, чтобы не перегрузить аналитиков ложными тревогами.
- **Выявление тонких угроз.** Многие современные атаки (например, некоторые виды DDoS, эксплойты нулевого дня) проявляются не в резких скачках средних значений, а в изменении формы распределения или в появлении новых паттернов в «хвостах». Методы, опирающиеся на анализ «хвостов», позволяют ловить такие угрозы, которые пропустила бы простая проверка на отклонение от среднего.
- **Адаптивность.** Реальная среда динамична: меняется профиль нагрузки, появляются новые сервисы. Если система умеет периодически перестраивать модель распределения (например, с помощью онлайн-алгоритмов оценки параметров), она остаётся эффективной дольше.
- **Выбор алгоритмов.** Знание распределения подсказывает, какие статистические или машинно-обучаемые методы применить.

Любая случайная величина (признак) может быть описана одним из типовых законов распределения. При этом необходимо понимать, что закон распределения не является фиксированным уравнением зависимости одной переменной от другой. Фиксируется только характер этой зависимости.

Законы распределения случайных величин делятся на **дискретные** и **непрерывные**.

В таблице приведены наиболее важные в кибербезопасности законы распределения.

Закон распределения	Тип признака	Пример в кибербезопасности	Как используется для детекции
Пуассона	Дискретный	Число запросов в минуту	Мониторинг резкого роста λ (DDoS, брутфорс)
Экспоненциальное	Непрерывный	Время между запросами	Выявление ботов (слишком малые интервалы)
Логнормальное / Парето	Непрерывный	Размер файлов, объём трафика	Поиск аномалий в «тяжёлых хвостах» (эксфильтрация)
Нормальное	Непрерывный	Загрузка CPU, джиттер сети	Правило 3σ для поиска отклонений от базовой линии
Равномерное	Дискретный/Непрерывный	Эфемерные порты, энтропия паролей	Поиск паттернов (не-случайности), указывающих на сканирование
Биномиальное	Дискретный	Факт блокировки запроса (0/1)	Расчёт вероятности успешной атаки при множестве попыток

Рассмотрим подходы к их оцениванию.

Способы оценивания закона распределения признака.

На практике для оценивания закона распределения признаков используются два способа.

1. Визуальный анализ. Для оценивания степени близости распределения признака одному из теоретических законов распределения можно провести визуальный сравнительный анализ эмпирического и предполагаемого теоретического распределений.

Алгоритм:

- по выборочным данным строятся графики эмпирических функций распределения или вероятности (плотности);
- по выборочным данным определяются оценки параметров предполагаемого распределения;
- осуществляется генерация теоретического распределения с этими параметрами;
- строятся графики теоретических функций распределения или вероятности (плотности);
- проводится сравнительный визуальный анализ полученных графиков и делается вывод.

2. Статистические критерии. Проверяются гипотезы о виде закона распределения признаков:

- критерий Колмогорова–Смирнова;
- критерий Андерсона–Дарлинга;
- критерий хи-квадрат;
- критерий Шапиро–Уилка.

Нормальное распределение

В «сыром» виде признаки в кибербезопасности редко бывают нормальными. Однако нормальное распределение возникает в трех ключевых сценариях:

- Агрегированные метрики системы (благодаря Центральной предельной теореме): Если сложить множество независимых мелких процессов, их сумма стремится к нормальному распределению.

Пример: загрузка CPU или оперативной памяти сервера в течение часа в штатном режиме. Она колеблется вокруг среднего значения (μ) с симметричным разбросом (σ).

- Сетевые задержки (Jitter) на стабильном канале:

Пример: время отклика (ping) до стабильного внутреннего сервера. Большинство значений группируются вокруг 2 мс, с редкими симметричными отклонениями до 2.5 мс или 1.5 мс из-за шума в сети.

- Ошибки измерений и шум сенсоров:

Пример: показания датчиков температуры в серверной или неточности в определении геолокации по IP.

Нормальное распределение (распределение Гаусса) — это фундамент статистики, но в кибербезопасности к нему нужно относиться с особой осторожностью. С одной стороны, это мощный инструмент для построения базовых профилей (baselining). С другой стороны, слепое предположение о нормальности сетевых данных — одна из самых частых причин низкого качества систем обнаружения аномалий (высокий уровень False Positive или False Negative).

Нормальное распределение относится к типу непрерывных.

Функция плотности распределения вероятностей: $f(x) = \frac{1}{\sigma\sqrt{2\pi}} \cdot e^{-\frac{(x-M(X))^2}{2\sigma^2}}$.

Функция распределения вероятностей: $F(X < x) = \int_{-\infty}^{+\infty} \frac{1}{\sigma\sqrt{2\pi}} \cdot e^{-\frac{(x-M(X))^2}{2\sigma^2}} dx$.

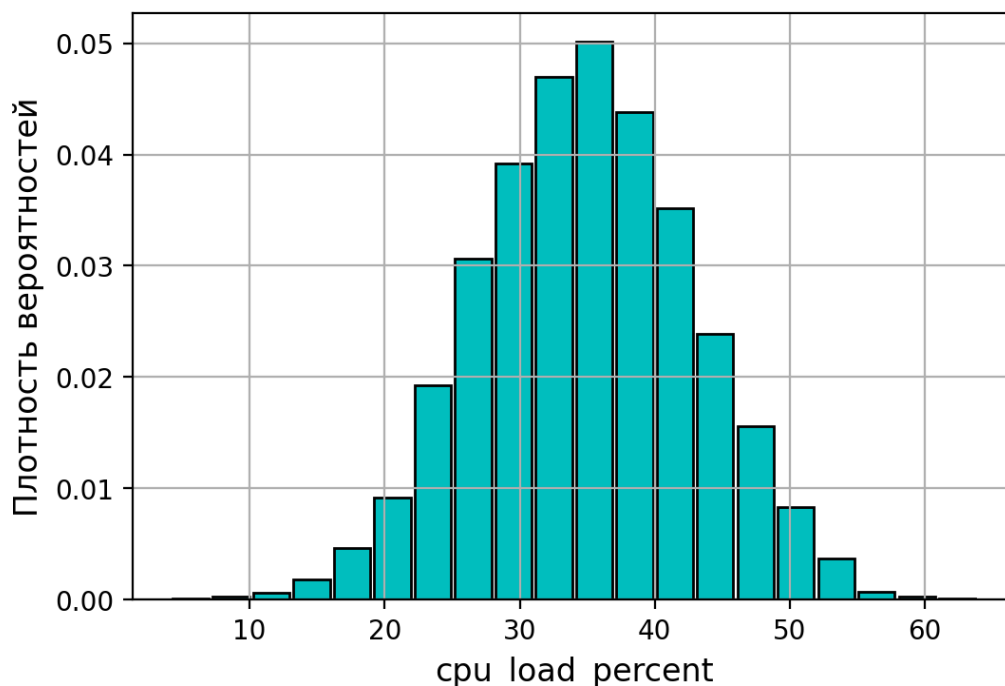
Параметры нормального закона: $M(x)$, σ

Визуальный анализ

Оценим визуальное соответствие распределения признака `cpu_load_percent` нормальному закону.

Построим гистограмму распределения признака, которая визуализирует эмпирическую функцию плотности распределения (EPDF).

```
In [26]: plt.figure(figsize = (6, 4))
plt.hist(df['cpu_load_percent'], color = 'c', bins = 20,
         density = True, edgecolor = 'black', rwidth = 0.9)
plt.xlabel('cpu_load_percent', fontsize = 12)
plt.ylabel('Плотность вероятностей', fontsize = 12)
plt.grid()
```



Для генерации случайных величин с различными законами распределения, а так же вычисления функций вероятностей (плотности) и функций распределения удобно использовать модуль `stats` библиотеки `scipy`.

```
In [27]: import scipy.stats as sts
```

Первоначально создается генератор случайной величины для выбранного закона распределения:

```
stats.имя_распределения(параметры_распределения) .
```

Для созданного генератора доступны следующие методы:

- `.rvs(size = N)` - генерация выборки с заданным законом распределения размера N. Когда вызывается `rvs` для конкретного распределения, метод возвращает массив типа `numpy.array` из N независимых значений, которые подчиняются этому распределению;
- `.cdf(x)` - значение функции распределения (CDF) в точке x;
- `.ppf(q)` - q-квантиль (Percent Point Function);
- `.mean()` - математическое ожидание;
- `.median()` - медиана;
- `.var()` - дисперсия;
- `.std()` - стандартное отклонение.

Кроме того для непрерывных распределений определена функция плотности (PDF) в точке x:

- `.pdf(x)` .

Для дискретных - функция вероятностей (PMF) в точке x:

- `.pmf(x)` .

Сформируем теоретическую функцию плотности распределения (PDF).

Найдем оценки параметров нормального закона по выборочным данным.

```
In [28]: m = df['cpu_load_percent'].mean()
s = df['cpu_load_percent'].std()
print(f'Математическое ожидание: {m}')
print(f'Стандартное отклонение: {s}')
```

Математическое ожидание: 34.92446556910189

Стандартное отклонение: 7.9555953318214785

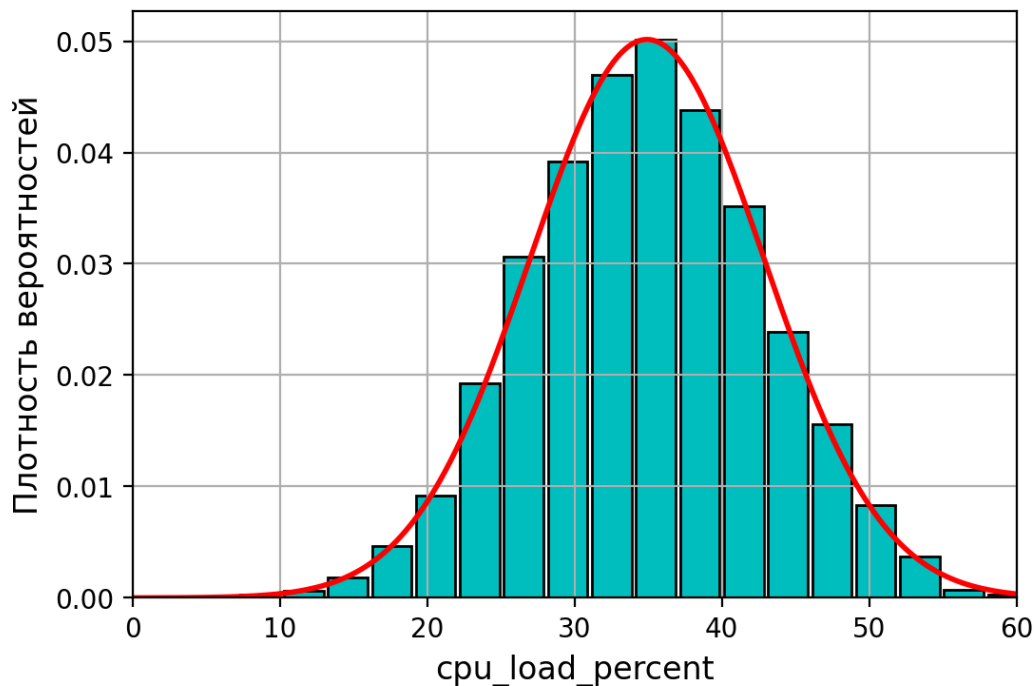
Вычислим значения функции плотности с найденными оценками параметров.

```
In [29]: # создание генератора с рассчитанными параметрами
norm_rv = sts.norm(loc = m, scale = s)
# генерация 1000 равномерно расположенных точек
x = np.linspace(0, 60, 1000)
# расчет значения функции плотности
norm_pdf = norm_rv.pdf(x)
norm_pdf[:10]
```

```
Out[29]: array([3.27715765e-06, 3.38749031e-06, 3.50133801e-06, 3.61880567e-06,
                3.74000114e-06, 3.86503521e-06, 3.99402173e-06, 4.12707764e-06,
                4.26432311e-06, 4.40588154e-06])
```

Добавим график полученной функции к гистограмме.

```
In [30]: plt.figure(figsize = (6, 4))
plt.hist(df['cpu_load_percent'],
         color = 'c',
         bins = 20, density = True,
         edgecolor = 'black',
         rwidth = 0.9)
plt.plot(x, norm_pdf, color = 'r', lw = 2)
plt.xlim(0, 60)
plt.xlabel('cpu_load_percent', fontsize = 12)
plt.ylabel('Плотность вероятностей', fontsize = 12)
plt.grid()
```

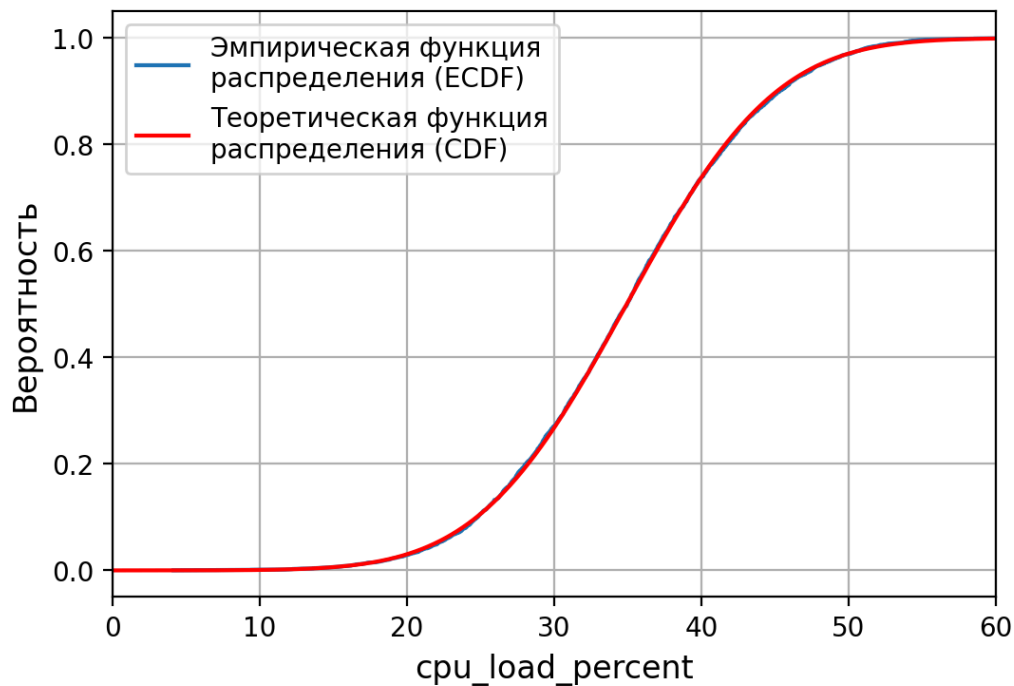


Так же можно провести сравнительный визуальный анализ графиков функций распределения вероятностей.

```
In [31]: # построение ECDF с использованием библиотеки statsmodels
from statsmodels.distributions.empirical_distribution import ECDF
plt.figure(figsize = (6, 4))

# эмпирическая функция распределения (ecdf)
ecdf = ECDF(df['cpu_load_percent'])
plt.plot(ecdf.x, ecdf.y,
         label = 'Эмпирическая функция\нраспределения (ECDF)')

# теоретическая функция распределения (cdf)
cdf = norm_rv.cdf(x)
plt.plot(x, cdf, color = 'r',
         label = 'Теоретическая функция\нраспределения (CDF)')
plt.xlim(0, 60)
plt.xlabel('cpu_load_percent', fontsize = 12)
plt.ylabel('Вероятность', fontsize = 12)
plt.legend()
plt.grid()
```



Таким образом, по результатам визуального анализа можно сделать вывод о нормальном распределении признака `cpu_load_percent`.

Статистические критерии

Для подтверждения результатов визуального анализа необходимо проверить статистические гипотезы о виде закона распределения признаков.

Для этого целесообразно использовать модуль `stats` библиотеки `scipy`.

Для проверки такого типа гипотез применяется класс критериев согласия. Их часть используется только для проверки на нормальный закон распределения, а остальные для проверки на любой предполагаемый закон.

Проверим статистическую гипотезу о распределении значений признака `cpu_load_percent` по нормальному закону:

H_0 : признак `cpu_load_percent` имеет нормальное распределение

H_1 : признак `cpu_load_percent` имеет распределение отличное от нормального.

Предпосылкой такого предположения являются результаты визуального анализа гистограммы.

Критерии нормальности.

```
In [32]: # критерий Шапиро-Уилка
import scipy.stats as stats
stat, p_value = stats.shapiro(df['cpu_load_percent'])
print(f"Критерий Шапиро-Уилка: p-value = {p_value:.4f}")

if p_value > 0.05:
    print("H0 принимается: распределение можно считать нормальным.")
else:
    print("H0 отвергается: распределение НЕ является нормальным")
```

Критерий Шапиро-Уилка: p-value = 0.5479
H0 принимается: распределение можно считать нормальным.

```
In [33]: # критерий Д'Агостино
stat, p_value = stats.normaltest(df['cpu_load_percent'])
print(f"Критерий критерий Д'Агостино: p-value = {p_value:.4f}")

if p_value > 0.05:
    print("H0 принимается: распределение можно считать нормальным.")
else:
    print("H0 отвергается: распределение НЕ является нормальным")
```

Критерий критерий Д'Агостино: p-value = 0.6201
H0 принимается: распределение можно считать нормальным.

Общие критерии.

```
In [34]: # критерий Андерсона-Дарлинга
result_ad = stats.anderson(df['cpu_load_percent'], dist='norm')
print(f"Критерий Андерсона-Дарлинга:")
print(f"Статистика: {result_ad.statistic:.4f}")
print("Критические значения:")
for i in range(len(result_ad.critical_values)):
    sig_level = result_ad.significance_level[i]
    crit_val = result_ad.critical_values[i]
    is_normal = "✅" if result_ad.statistic < crit_val else "❌"
    print(f" {sig_level}%: {crit_val:.4f} {is_normal}")
```

Критерий Андерсона-Дарлинга:

Статистика: 0.3533

Критические значения:

15.0%: 0.5760	✅
10.0%: 0.6550	✅
5.0%: 0.7860	✅
2.5%: 0.9170	✅
1.0%: 1.0910	✅

```
In [35]: # критерий Колмогорова-Смирнова
stat_ks_1, p_ks_1 = stats.kstest(df['cpu_load_percent'], 'norm')

print(f"D-статистика (макс. расстояние между кривыми): {stat_ks_1:.4f}")
print(f"p-value: {p_ks_1:.4e}")

if p_ks_1 > 0.05:
    print("H0 принимается: распределение можно считать нормальным.")
else:
    print("H0 отвергается: распределение НЕ является нормальным")
```

D-статистика (макс. расстояние между кривыми): 1.0000

p-value: 0.0000e+00

H0 отвергается: распределение НЕ является нормальным

При использовании статистических критериев нужно учитывать парадокс больших выборок: если $n = 100000$, то даже микроскопическое, незначимое для практики отклонение от идеального «колокола» (например, асимметрия 0.01 вместо 0) приведет к тому, что p-value станет меньше 0.0001.

Тест формально скажет: «распределение НЕ является нормальным». Но если оценить значения асимметрии (skewness) и эксцесса (kurtosis), то данные можно считать «практически нормальными» для применения параметрических методов (даже если строгий тест их отверг).

```
In [36]: # Проверка "практической" нормальности через моменты
skew = stats.skew(df['cpu_load_percent'])
kurt = stats.kurtosis(df['cpu_load_percent'])

print(f"Асимметрия (Skewness): {skew:.3f} (в идеале 0)")
print(f"Экцесс (Kurtosis): {kurt:.3f} (в идеале 0)")

if abs(skew) < 0.5 and abs(kurt) < 0.5:
    print("Данные достаточно близки к нормальному распределению.")
else:
    print("Распределение имеет заметную асимметрию или тяжелые хвосты.")
```

Асимметрия (Skewness): -0.004 (в идеале 0)

Экцесс (Kurtosis): -0.068 (в идеале 0)

Данные достаточно близки к нормальному распределению.

Если известен закон распределения признака, то можно решать задачи оценивания вероятности наступления различных событий.

Для этого используются свойства функции распределения:

$$P(X \leq b) = F(b) = \int_{-\infty}^b f(x)dx$$

$$P(a \leq X \leq b) = F(b) - F(a) = \int_a^b f(x)dx$$

$$P(b \geq X) = 1 - F(b)$$

Для расчета функции распределения в точке x используется метод `stats.norm.cdf(x, loc, scale)` библиотеки SciPy.

Оценим вероятность того, что значение признака `cpu_load_percent` (загрузка CPU в штатном режиме) не превышает 45%.

```
In [37]: p = round(stats.norm.cdf(45, loc = m, scale = s), 2)
print(f'Вероятность загрузки CPU менее чем на 45% = {p.round(2)}')
```

Вероятность загрузки CPU менее чем на 45% = 0.9

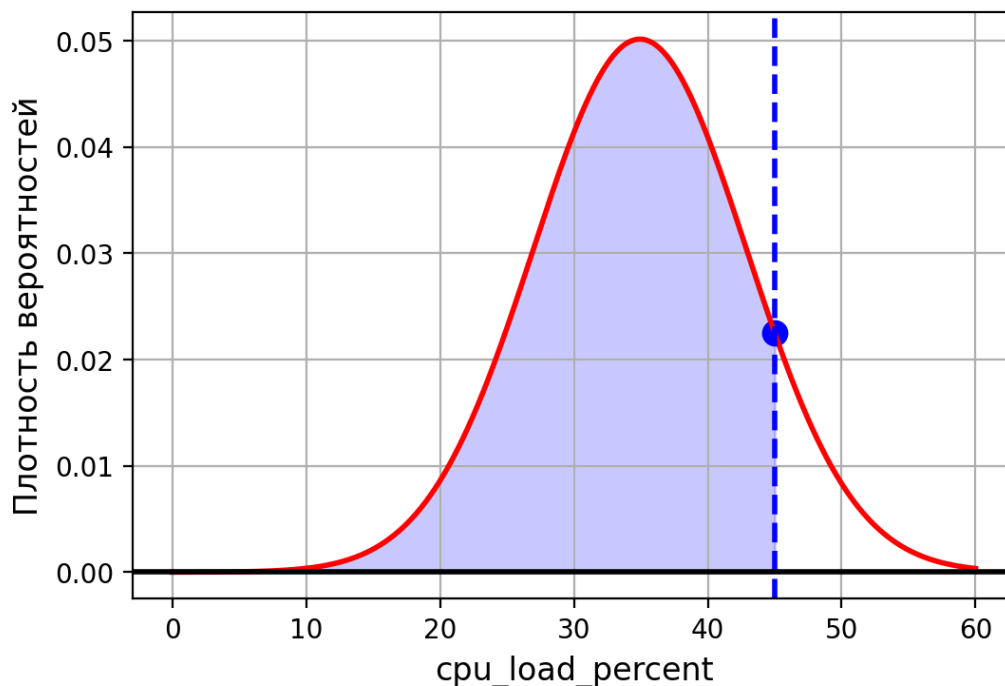
На графике функции плотности распределения эта вероятность соответствует площади фигуры, ограниченной осью абсцисс, линией $x = 45$ и кривой распределения:

$$P(X < 45) = F(45) = \int_{-\infty}^{45} f(x)dx \approx 0,9.$$

```
In [38]: plt.figure(figsize = (6, 4))
# равномерно распределенные точки
x = np.linspace(0, 60, 1000)
# расчет функции плотности распределения в каждой точке x
pdf = stats.norm.pdf(x, loc = m, scale = s)
# график теоретической функции плотности
plt.plot(x, pdf, lw = 2, color = 'r')
# отображение точки f(45)
plt.scatter([45], [stats.norm.pdf(45, loc = m, scale = s)],
            color = 'b', lw = 4)
# график линии x=45
plt.axvline(45, color = 'b', linestyle = "--", lw = 2)
# график линии y=0
plt.axhline(0, color = 'black', lw = 2)
# формирование данных для построения
```

```
# усеченного графика плотности до значения 45
xq = np.linspace(0, 45, 1000)
pdfq = sts.norm.pdf(xq, loc = m, scale = s)
plt.fill_between(xq, 0, pdfq, color = 'b', alpha = 0.2)

plt.xlabel('cpu_load_percent', fontsize=12)
plt.ylabel('Плотность вероятностей', fontsize = 12)
plt.grid()
```



Оценим вероятность того, что значение признака `cpu_load_percent` более 20%:

```
1-stats.norm.cdf(x, loc, scale)
stats.norm.sf(x, loc, scale)
```

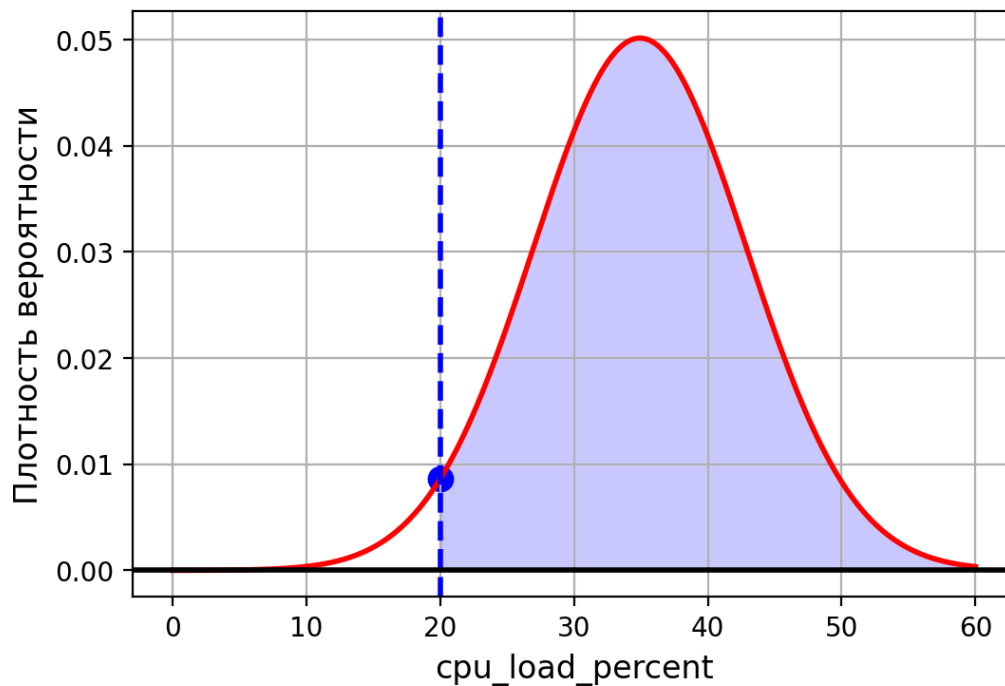
```
In [39]: p = 1 - sts.norm.cdf(20, loc = m, scale = s)
# Альтернатива
sts.norm.sf(20, loc = m, scale = s)
print(f'Вероятность загрузки CPU более чем на 20% = {p.round(2)}')
```

Вероятность загрузки CPU более чем на 20% = 0.97

```
In [40]: plt.figure(figsize=(6, 4))
plt.plot(x, pdf, lw = 2, color = 'r')
plt.scatter([20], [sts.norm.pdf(20, loc = m, scale = s)],
            color = 'b', lw=4)

plt.axvline(20, color = 'b', linestyle = "--", lw = 2)
plt.axhline(0, color = 'black', lw = 2)

xq = np.linspace(20, 60, 1000)
pdfq = sts.norm.pdf(xq, loc = m, scale = s)
plt.fill_between(xq, 0, pdfq, color = 'b', alpha = 0.2)
plt.xlabel('cpu_load_percent', fontsize=12)
plt.ylabel('Плотность вероятности', fontsize = 12)
plt.grid()
```



Оценим вероятность того, что значение признака `cpu_load_percent` находится в пределах от 25% до 45%:

```
stats.norm.sf(x1, loc, scale) - stats.norm.sf(x2, loc, scale), x1 > x2
```

```
In [41]: p = sts.norm.cdf(45, loc = m, scale = s) -\
          sts.norm.cdf(25, loc = m, scale = s)

print(f'Вероятность загрузки CPU в пределах от 25% до 45% = {p.round(2)}')
```

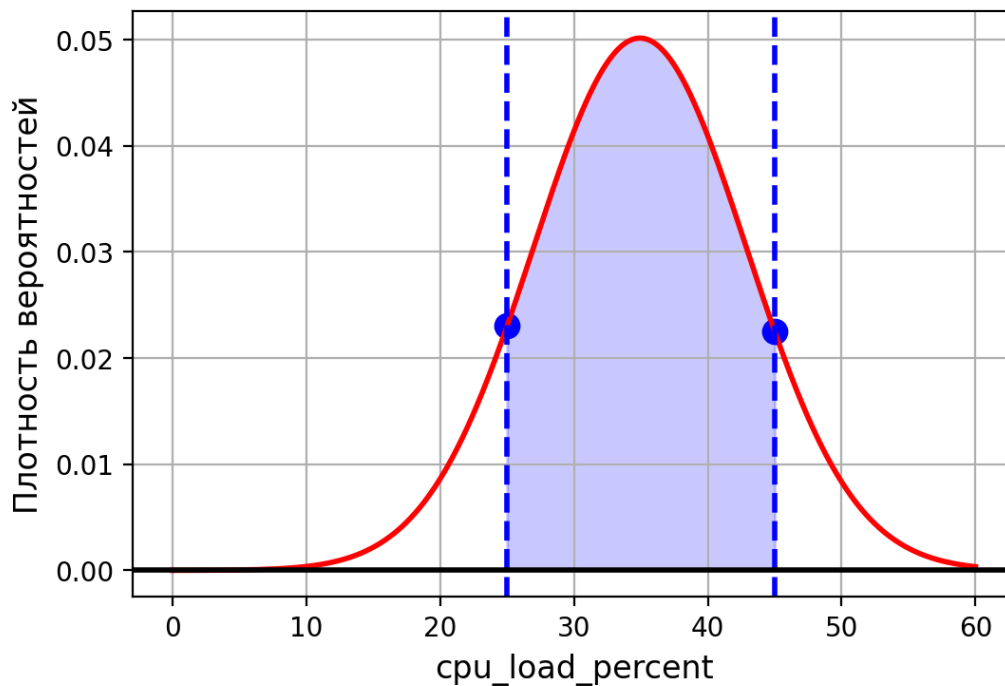
Вероятность загрузки CPU в пределах от 25% до 45% = 0.79

```
In [42]: plt.figure(figsize = (6, 4))
plt.plot(x, pdf, lw = 2, color = 'r')
plt.scatter([25, 45],
            [sts.norm.pdf(25, loc=m, scale=s),
             sts.norm.pdf(45, loc=m, scale=s)],
            color = 'b', lw = 4)

plt.axvline(45, color = 'b', linestyle = "--", lw = 2)
plt.axvline(25, color = 'b', linestyle = "--", lw = 2)
plt.axhline(0, color = 'black', lw = 2)

xq = np.linspace(25, 45, 1000)
pdfq = sts.norm.pdf(xq, loc = m, scale = s)
plt.fill_between(xq, 0, pdfq, color = 'b', alpha = 0.2)

plt.xlabel('cpu_load_percent', fontsize = 12)
plt.ylabel('Плотность вероятностей', fontsize = 12)
plt.grid()
```



Можно решать обратные задачи, когда по известной вероятности необходимо найти значение признака, т. е. вычислить квантиль распределения. Для этого используется метод `stats.norm.ppf(q, loc, scale)`.

Для признака `cpu_load_percent` определим квантиль уровня 0.9, т. е. найдем такое значение признака, меньше которого имеют 90% наблюдений.

```
In [43]: a = sts.norm.ppf(0.9, loc = m, scale = s)
print(f'90% наблюдений имеют значение признака менее {a.round(2)}%')
```

90% наблюдений имеют значение признака менее 45.12%

```
In [44]: df = df.drop(columns=['cpu_load_bin'])
```

Равномерное распределение

Равномерное распределение в кибербезопасности занимает уникальное место. В отличие от нормального распределения, которое описывает «шум» и естественные колебания, равномерное распределение часто является эталоном безопасности и криптографической стойкости.

В информационной безопасности работает фундаментальный принцип: чем ближе распределение данных к равномерному, тем выше его энтропия (непредсказуемость), и тем сложнее его взломать или предсказать.

Наиболее распространенные сценарии равномерного распределения в кибербезопасности.

1. Временные порты источника (Source Ports). Когда клиент (браузер, приложение) иницирует TCP/UDP-соединение, операционная система выбирает временный порт из диапазона эфемерных портов (обычно 49152 – 65535).

Норма: равномерное распределение - каждый порт имеет равную вероятность быть выбранным.

Атака: при сканировании портов (Port Scanning) или SYN-flood атаке злоумышленник часто использует последовательные порты (49152, 49153, 49154...) или фиксированный порт для всех запросов, чтобы упростить обработку ответов. Это резко нарушает равномерность.

2. Криптографические ключи, токены и пароли. Хороший генератор псевдослучайных чисел (ГПСЧ) должен выдавать байты, распределенные равномерно (каждый байт от 0x00 до 0xFF встречается с вероятностью 1/256).

Норма: равномерное распределение байтов в AES-ключе или JWT-токене.

Атака: уязвимость в ГПСЧ - распределение становится «дырявым» (появляются пики частот), что позволяет злоумышленнику перебрать возможные ключи.

3. Алгоритмы генерации доменов (DGA - Domain Generation Algorithms). Ботнеты используют DGA для создания тысяч доменных имен для связи с C&C-сервером. Чтобы обойти черные списки, злоумышленники пытаются сделать домены похожими на случайный набор символов.

Норма (для легитимных доменов): неравномерное распределение (в английском языке буквы 'e', 'a', 't' встречаются чаще, чем 'z', 'q', 'x').

Аномалия (DGA): злоумышленники пытаются имитировать равномерное распределение символов, но часто допускают ошибки (например, полное отсутствие гласных или равномерное распределение цифр и букв, что неестественно для человеческих слов). Анализ частоты символов (n-gram analysis) помогает детектировать такие домены.

Равномерное распределение может быть как дискретным, так и непрерывным.

При равномерном распределении вероятность наступления каждого исхода одинакова.

Нотация равномерного распределения:

$$X \sim U(a, b),$$

где a и b - минимальное и максимальное значения признака X .

Функция вероятности равномерного дискретного распределения (PMF):

$$p(x) = \frac{1}{n},$$

где $n = b - a + 1$.

Функция плотности вероятности равномерного непрерывного распределения (PDF):

$$f(x) = \frac{1}{b - a}.$$

Функция распределения равномерного дискретного распределения:

$$F(x) = \frac{[x] - a + 1}{b - a + 1},$$

где $[x]$ — это целая часть числа. Она возвращает наибольшее целое число, которое не больше x .

Функция распределения равномерного непрерывного распределения:

$$F(x) = \frac{x - a}{b - a}.$$

Математическое ожидание: $(a + b)/2$.

Дисперсия: $\frac{(b-a)^2}{12}$.

Оценим соответствие распределения дискретного признака `source_port` (номер порта на исходном устройстве) равномерному закону.

В случае большого количества уникальных значений признака целесообразно использовать непрерывный закон распределения.

```
In [45]: # Параметры равномерного распределения
a = df['source_port'].min()
b = df['source_port'].max()
width = b - a + 1

# Шаг 1: Визуальный анализ (Гистограмма + Теоретическая функция плотности PDF)

plt.figure(figsize=(8, 5))

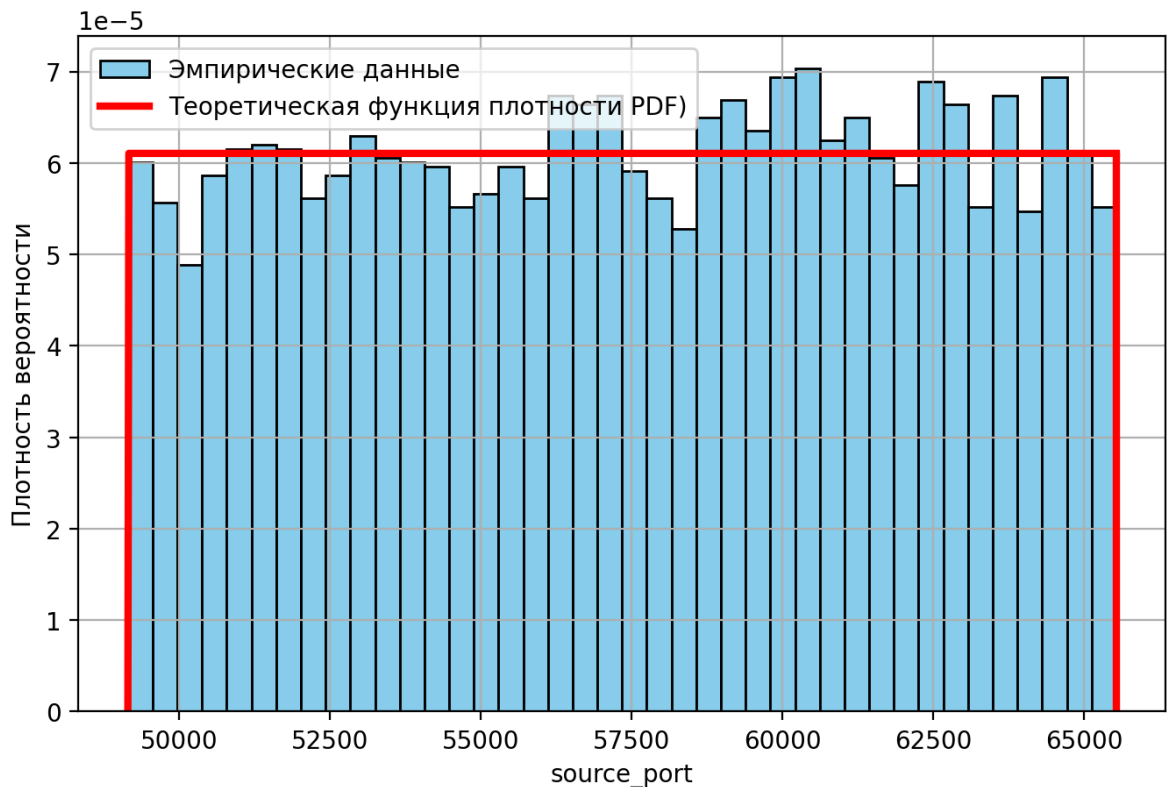
# 1. Гистограмма
counts, bins, patches = plt.hist(df['source_port'],
                                  bins=40,
                                  density=True,
                                  color='skyblue',
                                  edgecolor='black',
                                  label='Эмпирические данные')

# 2. Теоретическая функция плотности (PDF)
# В scipy.stats.uniform параметры: loc=a, scale=(b-a)
x_theor = np.linspace(a - 0.5, b + 0.5, 1000) # сетка для плавного графика
pdf_theor = stats.uniform.pdf(x_theor, loc=a, scale=width)

plt.plot(x_theor, pdf_theor, 'r-', linewidth=3,
         label=f'Теоретическая функция плотности PDF')

# Вертикальные линии на границах [a, b]
plt.vlines(b, ymin=0, ymax=pdf_theor.max(),
          color='red', linestyle='-', linewidth=3)

plt.xlabel('source_port')
plt.ylabel('Плотность вероятности')
plt.legend()
plt.grid()
```



Для формализованной проверки равномерности распределения в кибербезопасности используются два основных инструмента:

1. Энтропия Шеннона (Shannon Entropy).
2. Критерий согласия Пирсона (Хи-квадрат).

Энтропия Шеннона.

Отличный инструмент для проверки равномерности: чем ближе распределение к равномерному, тем выше энтропия, а максимум достигается именно при равномерности.

Для дискретной случайной величины с k возможными значениями и вероятностями $p_1, p_2, \dots, p_k$ энтропия (в битах) равна:

$$H(X) = - \sum_{i=1}^k p_i \cdot \log_2(p_i).$$

По соглашению: $0 \cdot \log_2(0) = 0$.

Максимально возможная энтропия для k значений достигается при равномерном распределении ($p_i = 1/k$) и равна: $H_{max} = \log_2(k)$.

На практике удобнее использовать нормированную энтропию (от 0 до 1):

$$\eta = \frac{H(X)}{H_{max}}$$

- $\eta \approx 1$: распределение почти равномерное (высокая неопределённость);
- $\eta \ll 1$: есть выраженные пики (низкая неопределённость, вероятны шаблоны/сканирование).

Рассмотрим пример использования энтропии Шенона для оценивания соответствия распределения признака `source_port` равномерному закону.

Для расчета энтропии Шенона можно использовать класс `entropy` модуля `stats` библиотеки `scipy`.

```
In [46]: from scipy.stats import entropy

# Получаем уникальные значения портов и сколько раз каждый встретился
# unique_vals – список уникальных портов, counts – сколько раз каждый порт встретился
unique_vals, counts = np.unique(df['source_port'], return_counts=True)

# Считаем вероятности: доля каждого порта в общем числе наблюдений
probs = counts / counts.sum()

# Вычисляем энтропию Шеннона в битах (base=2) по формуле  $H = -\sum(p * \log_2(p))$ 
H = entropy(probs, base=2)

# Определяем число реально встреченных уникальных портов в выборке
k_seen = len(unique_vals)

# Максимальная энтропия для k_seen значений достигается при равномерном распределении
# и равна  $\log_2(k\_seen)$ . Если порт всего один, энтропия 0, избегаем деления на 0
H_max_seen = np.log2(k_seen) if k_seen > 1 else 0.0

# Нормированная энтропия по реально увиденным портам
eta_seen = H / H_max_seen if H_max_seen > 0 else 0.0

# Задаём диапазон временных портов (стандартный диапазон для клиентских соединений)
eph_min, eph_max = 49152, 65535

# Общее число портов в этом диапазоне
k_eph = eph_max - eph_min + 1 # 16384

# Выделяем только временные порты из выборки
data_eph = df['source_port'][(df['source_port'] >= eph_min)
                             & (df['source_port'] <= eph_max)]

# Если в выборке есть временные порты, считаем для них отдельную энтропию
if len(data_eph) > 0:
    _, counts_eph = np.unique(data_eph, return_counts=True) # частоты временных портов
    probs_eph = counts_eph / counts_eph.sum() # их вероятности
    H_eph = entropy(probs_eph, base=2) # энтропия только по временным портам
    # Нормируем на полный диапазон эфемерных портов
    eta_eph = H_eph / np.log2(k_eph)
else:
    # Если временных портов нет, нормированная энтропия равна 0
    eta_eph = 0.0

# Выводим результаты для интерпретации
print(f"Всего наблюдений: {len(df['source_port'])}")
print(f"Уникальных портов в выборке: {k_seen}")
print(f"Энтропия (биты): {H:.3f}")
print(f"Нормированная энтропия по увиденным портам: {eta_seen:.3f}")
print(f"Нормированная энтропия временных портов (на диапазон 16384): {eta_eph:.3f}")
```

Всего наблюдений: 5000

Уникальных портов в выборке: 4263

Энтропия (биты): 11.979

Нормированная энтропия по увиденным портам: 0.993

Нормированная энтропия временных портов (на диапазон 16384): 0.856

Как использовать вычисленные метрики в детекции аномалий.

- `eta_seen` — (по реально увиденным портам) показывает, насколько равномерно используются именно те порты, которые встретились. Если 5000 соединений используют 50 портов, и каждый примерно по 100 раз — эта метрика будет близка к 1. Если используют 50 портов, но 90 % трафика на 3 из них — метрика резко упадёт. Это хорошо для детекции «перекоса» внутри текущего набора.
- `eta_eph` — (по полному диапазону временных портов) показывает, насколько «размазано» поведение по всему пространству. Если используются только 100 портов из 16 384 возможных, даже при равномерном распределении внутри этих 100, нормированная энтропия будет низкой, примерно $\log_2(100)/\log_2(16384) \approx 0,48$. Это хороший индикатор сканирования: злоумышленник редко использует весь диапазон.

Интерпретация: чем ближе `eta` к 1, тем ближе распределение к равномерному:

- `eta_seen` ~ 1: среди увиденных портов распределение равномерное (нет перекоса);
- `eta_eph` ~ 1: порты равномерно «размазаны» по всему диапазону временных.

Критерий хи-квадрат.

Критерий хи-квадрат (χ^2) проверяет гипотезу о том, что наблюдаемые частоты соответствуют некоторому теоретическому распределению. В нашем случае — равномерному (порты выбираются равномерно):

- H_0 : распределение портов равномерное.
- H_1 : распределение не равномерное (есть перекосы, шаблоны, сканирование и т. п.).

Формула статистики: $\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}$,

где:

- O_i - наблюдаемая частота i -го значения признака (сколько раз встретился каждый порт);
- E_i - ожидаемая частота появления значения признака при равномерном распределении;
- k - число уникальных значений признака (уникальных портов).

Ограничение применения критерия хи-квадрат.

Для корректной работы критерия желательно, чтобы $E_i \geq 5$.

Если это требование не выполняется, то рекомендуется разбить диапазон изменения признака (портов) на m равных интервалов (бинов) и считать частоты попаданий в каждый интервал.

Для вычисления критерия хи-квадрат можно использовать класс `chisquare` модуля `stats` библиотеки `scipy`.

```
In [47]: from scipy.stats import chisquare
# Выбираем число бинов m так, чтобы E_i было не меньше 5
N = 5000
m = 100
```

```

bin_edges = np.linspace(0, 65536, m + 1)

# Считаем, сколько наблюдений попало в каждый бин
counts_bins, _ = np.histogram(df['source_port'], bins=bin_edges)

# Ожидаемая частота в каждом бине при равномерности
E_bin = N / m

# Если какие-то бины пустые, chisquare всё равно сработает, но лучше проверить E_bin
if E_bin < 5:
    print(f"E_bin = {E_bin:.2f} < 5 — уменьшите число бинов m.")

chi2_bins, p_bins = chisquare(f_obs = counts_bins, f_exp = E_bin)
print(f"chi2 = {chi2_bins:.3f}, p-value = {p_bins:.4f}")

# Интерпретация
alpha = 0.05
if p_bins < alpha:
    print("Отвергаем H0: распределение портов НЕ равномерное.")
else:
    print("Нет оснований отвергать H0: данные согласуются с равномерным распределением")

```

chi2 = 15108.560, p-value = 0.0000

Отвергаем H0: распределение портов НЕ равномерное.

Полученный результат не согласуется с результатами применения энтропии Шенона.

Это обусловлено следующими причинами:

1. Реальное распределение не бывает идеально равномерным.
2. При количестве бинов, равном 100, ожидаемая частота в бине $E = 50$. Если в каком-то бине оказалось 30 вместо 50, это уже даёт вклад $\frac{(30-50)^2}{50} = 8$ в статистику. А если таких бинов несколько, то χ^2 быстро возрастает до тысяч.

На практике при использовании критерия хи-квадрат рекомендуется проверять равномерность только в «рабочем» поддиапазоне признака.

Если мы знаем, что легитимный трафик в сети использует только временные порты (49152–65535), то проверять равномерность нужно только там, а не по всему диапазону 0–65535.

```

In [48]: eph_min, eph_max = 49152, 65535
data_eph = df['source_port'][(df['source_port'] >= eph_min)
                             & (df['source_port'] <= eph_max)]

N_eph = len(data_eph)

m_eph = 100
bin_edges_eph = np.linspace(eph_min, eph_max + 1, m_eph + 1)
counts_eph, _ = np.histogram(data_eph, bins=bin_edges_eph)
E_eph = N_eph / m_eph

chi2_eph, p_eph = chisquare(f_obs=counts_eph, f_exp=E_eph)

print(f"chi2 = {chi2_eph:.3f}, p-value = {p_eph:.4f}")
alpha = 0.05
if p_eph < alpha:
    print("Отвергаем H0: распределение портов НЕ равномерное.")
else:
    print("Нет оснований отвергать H0: данные согласуются с равномерным распределением")

```

chi2 = 83.560, p-value = 0.8670

Нет оснований отвергать H0: данные согласуются с равномерным распределением.

Логнормальное распределение

Логнормальное распределение — это, пожалуй, самое важное и одновременно самое коварное распределение в кибербезопасности. Оно описывает явления с так называемыми «тяжелыми хвостами», где экстремальные события происходят гораздо чаще, чем предсказывает классическое нормальное распределение.

Случайная величина X имеет логнормальное распределение, если её натуральный логарифм распределен нормально.

Пусть $X \sim \text{Lognormal}$, тогда $X' = \ln(X) \sim \mathcal{N}(\mu, \sigma^2)$.

Параметры:

- μ - математическое ожидание $\ln(X)$ (не самого X);
- σ - стандартное отклонение $\ln(X)$.

Функция плотности вероятности (PDF): $f(x) = \frac{1}{x\sigma\sqrt{2\pi}} \exp\left(-\frac{(\ln x - \mu)^2}{2\sigma^2}\right)$, $x > 0$.

Функция распределения (CDF): $F(x) = \Phi\left(\frac{\ln x - \mu}{\sigma}\right)$.

Это означает, что X может принимать только положительные значения, а его график имеет резкий пик слева и длинный, пологий «хвост» вправо.

Типичные признаки в кибербезопасности:

- Размер полезной нагрузки (Payload Size) / Размер файлов: большинство HTTP-запросов или файлов маленькие (килобайты), но иногда передаются огромные объемы данных (гигабайты при эксфильтрации или загрузке вредоносного ПО).
- Время реакции системы / Задержки: в штатном режиме задержки малы, но из-за очередей, блокировок или атак (например, Slowloris) могут возникать редкие, но огромные всплески времени отклика.
- Финансовый ущерб от инцидентов кибербезопасности: большинство инцидентов стоят дешево, но единичные утечки данных обходятся компаниям в десятки миллионов рублей.

Главный трюк в проверке логнормальности: вместо того чтобы проверять X на логнормальность, мы берем натуральный логарифм $\ln(X)$ и проверяем его на нормальность. Это математически эквивалентно, но позволяет использовать мощные и хорошо изученные тесты для нормального распределения.

Рассмотрим пример оценивания соответствия распределения признака `payload_size_bytes` логнормальному закону.

Визуальный анализ.

```
In [49]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))  
  
# Гистограмма исходных данных (показывает "тяжелый хвост")
```

```

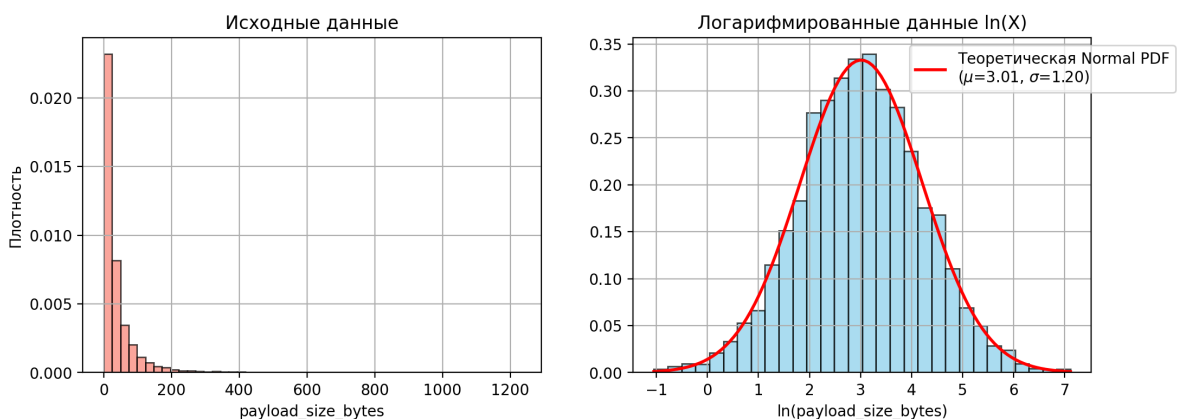
axes[0].hist(df['payload_size_bytes'], bins=50, density=True,
            color='salmon', edgecolor='black', alpha=0.7)
axes[0].set_title('Исходные данные')
axes[0].set_xlabel('payload_size_bytes')
axes[0].set_ylabel('Плотность')
axes[0].grid()

# Параметры логнормального распределения
log_payload = np.log(df['payload_size_bytes'])
mu_est = np.mean(log_payload)
sigma_est = np.std(log_payload, ddof=1)

# Гистограмма логарифмированных данных
axes[1].hist(log_payload, bins=30, density=True, color='skyblue',
            edgecolor='black', alpha=0.7)
axes[1].set_title('Логарифмированные данные ln(X)')
axes[1].set_xlabel('ln(payload_size_bytes)')

# Кривая нормального распределения
x_norm = np.linspace(log_payload.min(), log_payload.max(), 100)
y_norm = stats.norm.pdf(x_norm, mu_est, sigma_est)
axes[1].plot(x_norm, y_norm, 'r-', linewidth=2,
            label=f'Теоретическая Normal PDF\n(\mu={mu_est:.2f}, \sigma={sigma_e
axes[1].legend(loc='upper right', bbox_to_anchor=(1.2, 1.0))
axes[1].grid();

```



Статистическая проверка гипотез.

Если применить критерии нормальности к исходным значениям признака `payload_sizes`, то гарантированно будет получено $p\text{-value} < 0.0001$ и гипотезу о нормальности нужно будет отвергать. Это очевидно из-за длинного хвоста.

Правильно применять критерии к $\ln(X)$:

- H_0 : логарифм данных ($\ln(X)$) имеет нормальное распределение
- H_1 : логарифм данных ($\ln(X)$) имеет распределение отличное от нормального

```

In [50]: # Тест Шапиро-Уилка
shapiro_stat, shapiro_p = stats.shapiro(log_payload)
print(f"Тест Шапиро-Уилка (для ln(X)):")
print(f"Статистика: {shapiro_stat:.4f}, P-value: {shapiro_p:.4f}")
if shapiro_p > 0.05:
    print("H0 принимается. Данные ln(X) можно считать нормальными.")
    print("Вывод: Исходный признак payload_size является логнормальным.")
else:
    print("H0 отвергается. Распределение ln(X) значительно отличается от нормального.")

```

```
# Тест Колмогорова-Смирнова
ks_stat, ks_p = stats.ks_test(df['payload_size_bytes'], 'lognorm', args=(sigma_est, 0),
print(f"\n2. Тест Колмогорова-Смирнова (для исходного X):")
print(f"Статистика D: {ks_stat:.4f}, P-value: {ks_p:.4f}")
if ks_p > 0.05:
    print("H0 принимается. Эмпирическое распределение согласуется с теоретическим логнормальным.")
else:
    print("H0 отвергается. Есть статистически значимые отклонения от логнормального з
```

Тест Шапиро-Уилка (для $\ln(X)$):

Статистика: 0.9996, P-value: 0.3858

H0 принимается. Данные $\ln(X)$ можно считать нормальными.

Вывод: Исходный признак `payload_size` является логнормальным.

2. Тест Колмогорова-Смирнова (для исходного X):

Статистика D: 0.0099, P-value: 0.7091

H0 принимается. Эмпирическое распределение согласуется с теоретическим логнормальным.

Экспоненциальное распределение

Экспоненциальное распределение описывает время между событиями в простейшем потоке (процессе Пуассона), где события происходят непрерывно, независимо друг от друга и с постоянной средней интенсивностью.

Типичные признаки в кибербезопасности:

- Время между запросами (Inter-arrival time): интервалы между легитимными HTTP-запросами от разных пользователей или между пакетами в стабильном сетевом потоке.
- Время между неудачными попытками входа: если пользователи забывают пароли случайно и независимо друг от друга.
- Время отклика системы (Response time): в простых системах массового обслуживания без сложных очередей.

Пусть X — время между событиями.

- Параметр: λ (лямбда) — интенсивность (среднее количество событий в единицу времени). Важно: в библиотеке `scipy` используется параметр масштаба `scale`, который равен $\theta = 1/\lambda$ (среднее время между событиями).
- Функция плотности вероятности (PDF): $f(x) = \begin{cases} \lambda e^{-\lambda x}, & x \geq 0, \\ 0, & x < 0. \end{cases}$
- Функция распределения (CDF): (вероятность того, что время ожидания будет $\leq x$): $F(x) = \begin{cases} 1 - e^{-\lambda x}, & x \geq 0, \\ 0, & x < 0. \end{cases}$
- Числовые характеристики:
 - математическое ожидание: $M(X) = \frac{1}{\lambda}$;
 - дисперсия: $D(X) = \frac{1}{\lambda^2}$.

Рассмотрим пример оценивания соответствия распределения признака

`response_time_ms` (время отклика системы) экспоненциальному закону.

Визуальный анализ.

```
In [51]: fig, axes = plt.subplots(figsize=(6, 4))
```

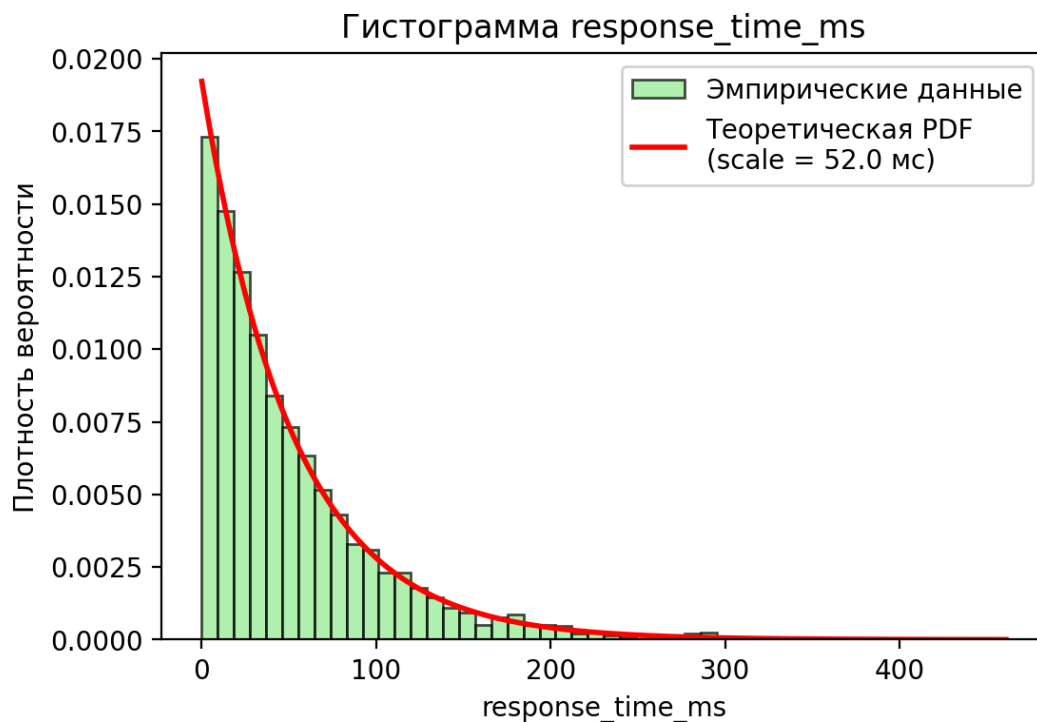
```

# Оцениваем параметр scale (среднее значение)
estimated_scale = np.mean(df['response_time_ms'])

# Гистограмма и теоретическая PDF
x_theory = np.linspace(0, df['response_time_ms'].max(), 500)
# В scipy: loc=0 (сдвиг, время не может быть < 0), scale=estimated_scale
y_theory = stats.expon.pdf(x_theory, loc=0, scale=estimated_scale)

axes.hist(df['response_time_ms'], bins=50, density=True, color='lightgreen',
          edgcolor='black', alpha=0.7, label='Эмпирические данные')
axes.plot(x_theory, y_theory, 'r-', linewidth=2,
          label=f'Теоретическая PDF\n(scale = {estimated_scale:.1f} мс)')
axes.set_title('Гистограмма response_time_ms')
axes.set_xlabel('response_time_ms')
axes.set_ylabel('Плотность вероятности')
axes.legend();

```



Статистическая проверка гипотез.

```

In [52]: # Тест Колмогорова-Смирнова (K-S test)
ks_stat, ks_p = stats.ks test(df['response_time_ms'], 'expon', args=(0, estimated_scale))

print(f"Тест Колмогорова-Смирнова:")
print(f"Статистика D: {ks_stat:.4f}")
print(f"P-value: {ks_p:.4f}")

alpha = 0.05
if ks_p > alpha:
    print(f"Н0 принимается (p-value > {alpha}). Данные согласуются с экспоненциальным")
else:
    print(f"Н0 отвергается (p-value < {alpha}). Распределение значимо отличается от э

# Тест Андерсона-Дарлинга (более чувствителен к хвостам распределения)
ad_result = stats.anderson(df['response_time_ms'], dist='expon')

print(f"\n2. Тест Андерсона-Дарлинга:")
print(f"Статистика: {ad_result.statistic:.4f}")
print("Критические значения:")
for i in range(len(ad_result.critical_values)):

```

```
sig_level = ad_result.significance_level[i]
crit_val = ad_result.critical_values[i]
is_exp = "✅" if ad_result.statistic < crit_val else "❌"
print(f"{sig_level}%: {crit_val:.4f} {is_exp}")
```

Тест Колмогорова-Смирнова:

Статистика D: 0.0074

P-value: 0.9478

H0 принимается (p-value > 0.05). Данные согласуются с экспоненциальным законом.

2. Тест Андерсона-Дарлинга:

Статистика: 0.2164

Критические значения:

15.0%: 0.9220 ✅

10.0%: 1.0780 ✅

5.0%: 1.3410 ✅

2.5%: 1.6060 ✅

1.0%: 1.9570 ✅

Распределение Пуассона

Распределение Пуассона моделирует вероятность наступления определенного количества независимых событий за фиксированный интервал времени (или пространства), если эти события происходят с известной постоянной средней интенсивностью.

Типичные признаки в кибербезопасности:

- Количество запросов в минуту: базовая активность легитимного пользователя или сервера.
- Число неудачных попыток входа: в штатном режиме пользователи иногда ошибаются.
- Количество срабатываний правил WAF/IDS в час.
- Число новых подключений к порту в секунду.

Распределение Пуассона относится к дискретным распределениям. Его целесообразно использовать для описания дискретных признаков, полигон распределения которых по виду близок к нормальному распределению.

Распределение Пуассона описывает вероятность получения k успехов за заданный интервал времени t .

Пусть X — количество событий за интервал.

- Параметр: λ - среднее количество успехов за определенный интервал (интенсивность).

- Функция вероятности PMF, т. е. вероятность того, что $X = k$:

$$P_t(X = k) = \frac{(\lambda \cdot t)^k}{k!} \cdot e^{-\lambda \cdot t}, \quad k - \text{количество успехов.}$$

- Функция распределения вероятностей: $F(k) = \sum_{i=1}^k \frac{(\lambda \cdot t)^i}{i!} \cdot e^{-\lambda \cdot t}$.

- Математическое ожидание (среднее количество событий за фиксированный промежуток времени): $M(k) = \lambda$.

- Дисперсия: $D(k) = \lambda$.

При увеличении λ распределение стремится к нормальному $N(\lambda, \sqrt{\lambda})$.

Рассмотрим пример оценивания соответствия распределения признака `requests_per_min` (количество запросов в минуту) закону Пуассона.

Визуальный анализ.

Для визуализации распределения необходимо использовать столбчатую диаграмму (`plt.bar`), а не гистограмму, потому что признак `requests_per_min` — это дискретная величина. Красные точки показывают теоретические вероятности. Идеальное наложение красных точек на вершины синих столбцов — визуальное подтверждение гипотезы.

Прежде чем запускать сложные тесты, рекомендуется проверять фундаментальное свойство распределения Пуассона - равенство математического ожидания и дисперсии.

```
In [53]: # Оценка параметра Lambda по выборке (среднее значение)
lambda_est = np.mean(df['requests_per_min'])
variance_est = np.var(df['requests_per_min'], ddof=1)

print(f"Оценка параметра  $\lambda$  (среднее): {lambda_est:.2f}")
print(f"Оценка дисперсии: {variance_est:.2f}")
print(f"Проверка свойства Пуассона (Среднее  $\approx$  Дисперсия): {'Да' if abs(lambda_est - v
print("-" * 60)

# Визуальный анализ (Столбчатая диаграмма + Теоретическая PMF)
# Для дискретных данных используем bar chart, а не гистограмму
# Получаем уникальные значения и их наблюдаемые частоты
unique_vals, obs_counts = np.unique(df['requests_per_min'], return_counts=True)

# Вычисляем теоретические вероятности и ожидаемые частоты
theoretical_probs = stats.poisson.pmf(unique_vals, lambda_est)
expected_counts = len(df['requests_per_min']) * theoretical_probs

plt.figure(figsize=(6, 4))

# Эмпирические данные
plt.bar(unique_vals, obs_counts / len(df['requests_per_min']), width=0.8,
        color='skyblue', edgecolor='black', alpha=0.7,
        label='Эмпирическая относительная частота')

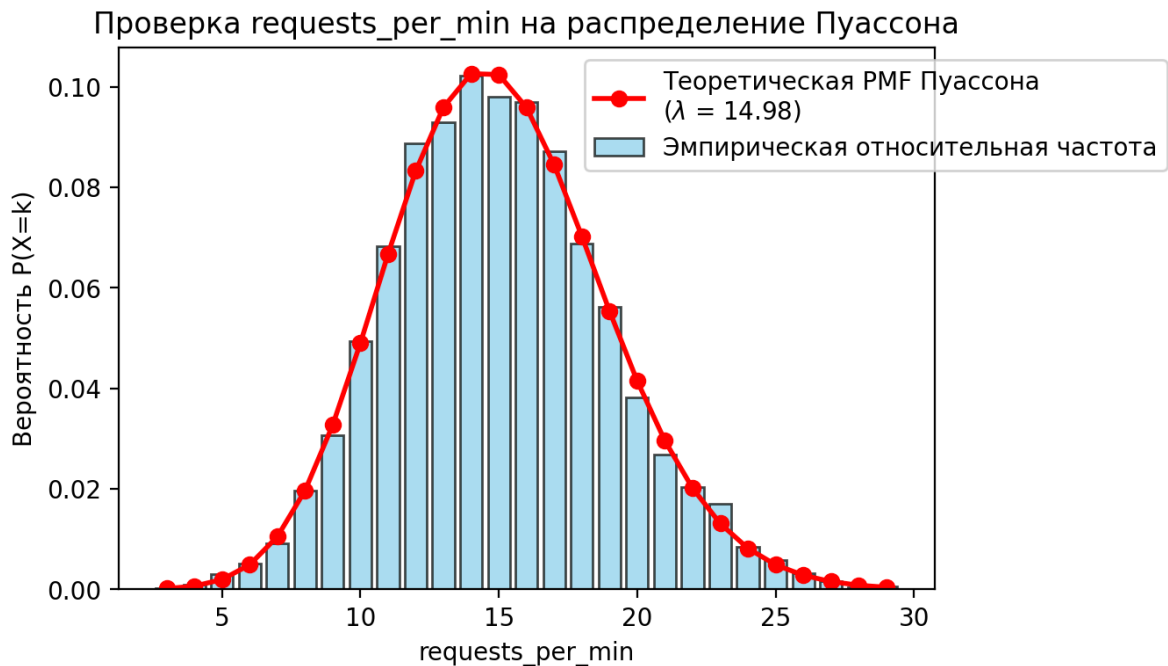
# Теоретическая функция вероятности
plt.plot(unique_vals, theoretical_probs, 'ro-', linewidth=2, markersize=6,
         label=f'Теоретическая PMF Пуассона\n( $\lambda$  = {lambda_est:.2f})')

plt.title('Проверка requests_per_min на распределение Пуассона')
plt.xlabel('requests_per_min')
plt.ylabel('Вероятность P(X=k)')
plt.legend(loc='upper right', bbox_to_anchor=(1.3, 1.0));
```

Оценка параметра λ (среднее): 14.98

Оценка дисперсии: 15.16

Проверка свойства Пуассона (Среднее $\approx$ Дисперсия): Да



Статистическая проверка гипотез.

Основным формальным тестом является критерий хи-квадрат (χ^2).

```
In [54]: # Для корректного chi-square теста ожидаемые частоты должны быть >= 5.
# Мы группируем "хвост" распределения (значения >= 30) в одну категорию.
max_val_to_check = 30

# Создаем бины.
# Добавляем np.inf в конец bins. Это гарантирует, что np.histogram НЕ потеряет значения
# Теперь бины выглядят так: [0, 1), [1, 2), ..., [29, 30), [30, infinity)
bins = list(range(max_val_to_check)) + [np.inf]

# Наблюдаемые частоты: np.histogram автоматически суммирует всё, что >= 30, в последний
observed_freq, _ = np.histogram(df['requests_per_min'], bins=bins)

# Вычисляем теоретические ВЕРОЯТНОСТИ
expected_probs = np.zeros(len(bins) - 1)
# Для всех бинов, кроме последнего, используем PMF
for i in range(len(bins) - 2):
    expected_probs[i] = stats.poisson.pmf(i, lambda_est)
# Для последнего бина (>= 30) используем 1 - CDF(29)
# (Вероятность того, что X >= 30)
expected_probs[-1] = 1 - stats.poisson.cdf(max_val_to_check - 1, lambda_est)

# Принудительная нормализация вероятностей.
# Делим каждый элемент на сумму всех элементов, чтобы избавиться от ошибок float.
# Теперь сумма expected_probs математически строго равна 1.0.
expected_probs = expected_probs / expected_probs.sum()

# Переводим вероятности в ожидаемые частоты
expected_freq = len(df['requests_per_min']) * expected_probs

# Проверка условия валидности Хи-квадрата (ожидаемые частоты >= 5)
print(f"Минимальная ожидаемая частота в бинах: {expected_freq.min():.2f}")
if expected_freq.min() < 5:
    print("ВНИМАНИЕ: В некоторых бинах ожидаемая частота < 5. Результат теста может б

# Проверяем равенство сумм наблюдаемых и ожидаемых частот
print(f"Сумма наблюдаемых частот: {observed_freq.sum()}")
```

```

print(f"Сумма ожидаемых частот: {expected_freq.sum():.2f}")

# Применяем критерий Хи-квадрат
# ddof=1, так как мы оценили 1 параметр (lambda) из самих данных
chi2_stat, p_value = stats.chisquare(f_obs=observed_freq, f_exp=expected_freq, ddof=1)

print(f"Статистика  $\chi^2$ : {chi2_stat:.4f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value > alpha:
    print(f"\nВЫВОД: P-value ({p_value:.4f}) > {alpha}. Мы НЕ ИМЕЕМ оснований отвергн")
    print("    Легитимный трафик статистически не отличим от распределения Пуассона.")
else:
    print(f"\nВЫВОД: P-value < {alpha}. Отвергаем H0.")
    print("    Распределение значимо отличается от Пуассона (возможно, присутствует ск

```

Минимальная ожидаемая частота в бинах: 0.00

ВНИМАНИЕ: В некоторых бинах ожидаемая частота < 5. Результат теста может быть неточны
м.

Сумма наблюдаемых частот: 5000

Сумма ожидаемых частот: 5000.00

Статистика χ^2 : 20.6523

P-value: 0.8395

ВЫВОД: P-value (0.8395) > 0.05. Мы НЕ ИМЕЕМ оснований отвергнуть H0.

Легитимный трафик статистически не отличим от распределения Пуассона.

Чтобы удовлетворить требование "ожидаемая частота ≥ 5 ", необходимо объединить
(сгруппировать) редкие значения в начале и в конце распределения в широкие бины,
пока их суммарная ожидаемая частота не превысит 5.

- Левый "хвост": объединим значения от 0 до 7 в один бин.
- Середина: оставим значения от 8 до 23 как отдельные бины (там частоты большие).
- Правый "хвост": объединим значения от 24 до $+\infty$ в один бин.

```

In [55]: # 1. Формируем "умные" бины для группировки хвостов
# [0, 8) -> включает 0, 1, 2, 3, 4, 5, 6, 7
# [8, 9), [9, 10), ..., [23, 24) -> отдельные значения
# [24, inf) -> включает 24, 25, 26, ... до бесконечности
bins = [0, 8] + list(range(9, 25)) + [np.inf]

# 2. Наблюдаемые частоты
observed_freq, _ = np.histogram(df['requests_per_min'], bins=bins)

# 3. Вычисляем теоретические вероятности для наших кастомных бинов
expected_probs = np.zeros(len(bins) - 1)

# Бин 0: Вероятность  $X \leq 7$ 
expected_probs[0] = stats.poisson.cdf(7, lambda_est)

# Средние бины: Вероятность  $X == k$  (для k от 8 до 23)
for i in range(1, len(bins) - 2):
    k = bins[i] # Это значения 8, 9, ..., 23
    expected_probs[i] = stats.poisson.pmf(k, lambda_est)

# Последний бин: Вероятность  $X \geq 24$ 
expected_probs[-1] = 1 - stats.poisson.cdf(23, lambda_est)

# 4. Принудительная нормализация (защита от float-ошибок)
expected_probs = expected_probs / expected_probs.sum()
expected_freq = len(df['requests_per_min']) * expected_probs

```

```

# 5. Проверка условия валидности
print(f"Минимальная ожидаемая частота в бинах: {expected_freq.min():.2f} (Должно быть >= 5)")
print(f"Сумма наблюдаемых частот: {observed_freq.sum()}")
print(f"Сумма ожидаемых частот: {expected_freq.sum():.2f}")
print("-" * 60)

# 6. Применяем критерий Хи-квадрат
# ddof=1, так как мы оценили 1 параметр (lambda) из данных
chi2_stat, p_value = stats.chisquare(f_obs=observed_freq, f_exp=expected_freq, ddof=1)

print(f"Статистика  $\chi^2$ : {chi2_stat:.4f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value > alpha:
    print(f"\nВывод: P-value ({p_value:.4f}) > {alpha}. Мы НЕ ИМЕЕМ оснований отвергнуть H0.")
    print("    Легитимный трафик статистически не отличим от распределения Пуассона.")
    print("    Предупреждений о малых частотах нет: анализ математически корректен.")
else:
    print(f"\nВывод: P-value < {alpha}. Отвергаем H0.")

```

Минимальная ожидаемая частота в бинах: 65.54 (Должно быть ≥ 5)

Сумма наблюдаемых частот: 5000

Сумма ожидаемых частот: 5000.00

Статистика χ^2 : 13.4574

P-value: 0.6391

Вывод: P-value (0.6391) > 0.05. Мы НЕ ИМЕЕМ оснований отвергнуть H0.

Легитимный трафик статистически не отличим от распределения Пуассона.

Предупреждений о малых частотах нет: анализ математически корректен.

Практическое применение в кибербезопасности.

Мы доказали, что легитимный трафик подчиняется закону Пуассона с $\lambda \approx 15$. Это позволяет математически точно рассчитать пороги для WAF (Web Application Firewall — специализированный межсетевой экран).

Какова вероятность, что легитимный пользователь сделает ≥ 30 запросов в минуту?

$$P(X \geq 25) = 1 - P(X \leq 25) = 1 - \text{stats.poisson.cdf}(25, 15) \approx 0.06\%.$$

Вывод для настройки: если установить правило "блокировать при >25 запросах в минуту", то получим частоту ложных срабатываний (False Positive Rate) всего 0.6%, при этом эффективно отсекая простейшие скрипты перебора, которые генерируют 50-100 запросов в минуту.

Этот подход превращает настройку безопасности из "угадывания чисел" в строгий, математически обоснованный инженерный процесс.

```
In [56]: print(f'Вероятность более 25 запросов = {(1-stats.poisson.cdf(25,15))*100}%')
```

Вероятность более 25 запросов = 0.6184903811266795%

Биномиальное распределение

Эксперименты, в которых результаты отражают состояние альтернативного признака — либо «да» (событие наступило), либо «нет» (событие не наступило), называются схемой Бернулли.

На практике исследователя интересует вероятность наступления некоторого числа успешных событий.

Биномиальное распределение моделирует количество «успехов» (интересующих нас событий) k в серии из n независимых испытаний, где вероятность «успеха» в каждом испытании постоянна и равна p .

Типичные признаки в кибербезопасности:

- Целевая переменная (`is_malicious`): в пакете из n запросов количество вредоносных.
- Процент успешных подборов пароля: из n попыток входа, сколько были успешными.
- Доставка фишинговых писем: из n отправленных писем, сколько пользователи открыли/перешли по ссылке.
- Потеря пакетов: количество потерянных пакетов из n отправленных через ненадежный канал.
- Параметры:
 - n — количество испытаний (размер выборки);
 - p — вероятность «успеха» в одном испытании.
- Функция вероятности (PMF) (вероятность того, что в n испытаниях произойдет ровно k подсчитываемых событий):

$$P_n(B = k) = C_n^k \cdot p^k \cdot (1 - p)^{n-k},$$

где k - количество успехов.

- Функция распределения:

$$F(B < k) = \sum_{i=1}^k C_n^i \cdot p^i \cdot (1 - p)^{n-i}.$$

- Числовые характеристики:
 - математическое ожидание: $M = p \cdot n$;
 - дисперсия: $D = n \cdot p \cdot (1 - p)$.

Рассмотрим пример оценивания соответствия распределения признака `is_malicious` (является ли событие вредоносным) биномиальному закону.

При проверке признака на биномиальное распределение, мы сталкиваемся с фундаментальной проблемой: столбец `is_malicious` состоит из отдельных нулей и единиц. Это распределение Бернулли (частный случай биномиального при $n = 1$). Критерии согласия (например, Хи-квадрат) плохо работают с двумя категориями, если мы не агрегируем данные.

Визуальный анализ.

Признак `is_malicious` для одной строки датасета может быть только 0 или 1. Это чистое распределение Бернулли.

```
In [57]: # Поскольку признак принимает только значения 0 и 1,  
# мы визуализируем его как распределение Бернулли (частный случай биномиального при n
```

```

p_observed = is_malicious.mean()
k_values = [0, 1] # Возможные исходы: 0 (легитимный), 1 (вредоносный)

# Эмпирические вероятности
empirical_probs = [(is_malicious == 0).mean(), # P(X=0)
                   (is_malicious == 1).mean()] # P(X=1)

# Теоретические вероятности Бернулли с параметром p = p_observed
theoretical_probs = [stats.binom.pmf(0, n=1, p=p_observed),
                     stats.binom.pmf(1, n=1, p=p_observed)]

plt.figure(figsize=(6, 4))

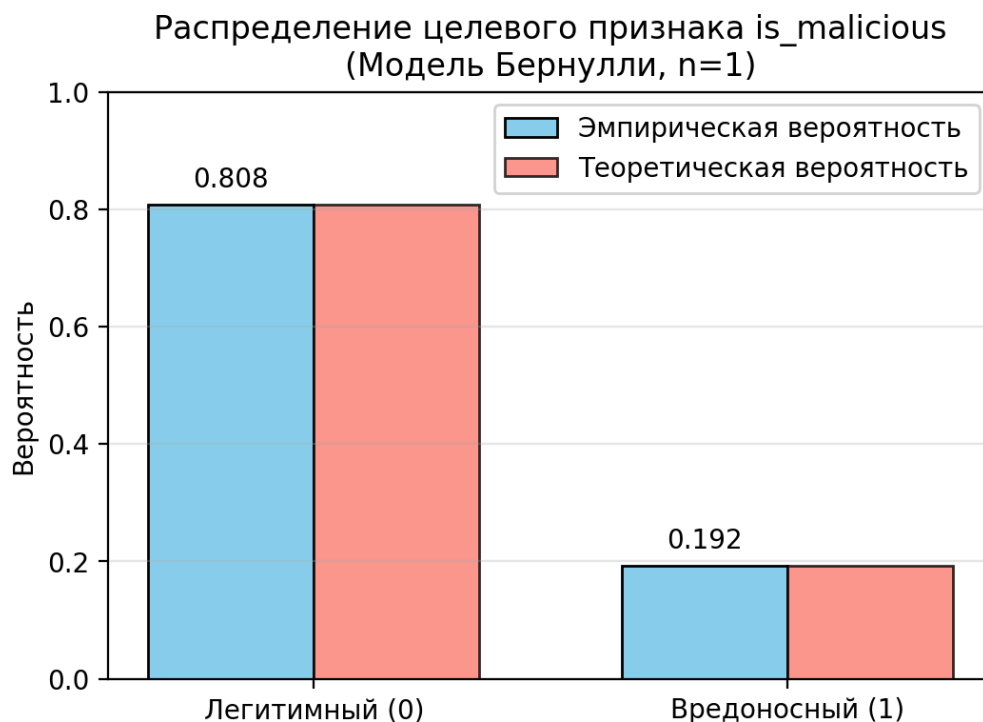
# Строим столбчатую диаграмму
x = np.arange(len(k_values))
width = 0.35
bars1 = plt.bar(x - width/2, empirical_probs, width,
                label='Эмпирическая вероятность', color='skyblue', edgecolor='black')
bars2 = plt.bar(x + width/2, theoretical_probs, width,
                label=f'Теоретическая вероятность', color='salmon', edgecolor='black')

plt.xticks(x, ['Легитимный (0)', 'Вредоносный (1)'])
plt.ylabel('Вероятность')
plt.title('Распределение целевого признака is_malicious\n(Модель Бернулли, n=1)')
plt.legend()
plt.ylim(0, 1.0)

# Добавляем значения на столбцы
for bar in bars1:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.02,
             f'{yval:.3f}', ha='center', va='bottom')

plt.grid(True, alpha=0.3, axis='y');

```



На диаграмме видно, что эмпирическая доля практически идеально совпадает с теоретической вероятностью Бернулли, рассчитанной по среднему значению выборки.

Это подтверждает, что на микроуровне данные ведут себя как серия независимых испытаний.

Чтобы проверить именно Биномиальный закон, мы должны искусственно сгруппировать данные в «пакеты» (например, по $n = 50$ запросов), посчитать количество «успехов» (k) в каждом пакете, и уже эту новую величину k проверять на соответствие $B(n, p)$.

Агрегация данных (превращение распределения Бернулли в Биномиальное).

```
In [58]: # Разбиваем поток запросов на пакеты размером n_batch
n_batch = 50
num_batches = n_samples // n_batch # 100 пакетов

# Преобразуем серию в массив
is_malicious = df['is_malicious'].to_numpy()

# Обрезаем массив до кратного размера и reshape в матрицу (num_batches x n_batch)
batched_data = is_malicious[:num_batches * n_batch].reshape(num_batches, n_batch)

# Считаем количество вредоносных запросов (k) в каждом пакете
k_observed = batched_data.sum(axis=1)

# Оцениваем параметр p по агрегированным данным
p_hat = k_observed.mean() / n_batch

print(f"Размер пакета (n): {n_batch}")
print(f"Количество пакетов (N): {num_batches}")
print(f"Оцененная вероятность успеха (p_hat): {p_hat:.4f}")
```

Размер пакета (n): 50

Количество пакетов (N): 100

Оцененная вероятность успеха (p_hat): 0.1920

Визуальный анализ соответствия агрегированного признака биномиальному распределению.

```
In [59]: # Возможные значения k: от 0 до n_batch
k_values = np.arange(n_batch + 1)

# Теоретические вероятности и ожидаемые частоты
theoretical_probs = stats.binom.pmf(k_values, n=n_batch, p=p_hat)
expected_freq = num_batches * theoretical_probs

# Наблюдаемые частоты (добавляем np.inf, чтобы захватить все значения до n_batch вклю
bins = np.arange(n_batch + 2) # [0, 1, 2, ..., n_batch, n_batch+1]
obs_freq, _ = np.histogram(k_observed, bins=bins)

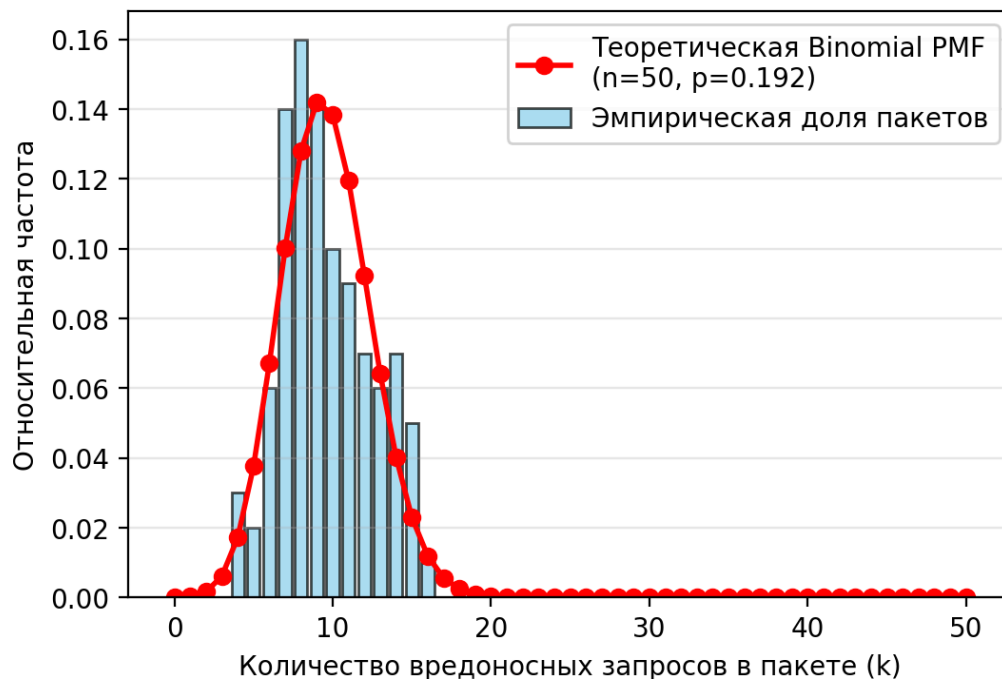
plt.figure(figsize=(6, 4))

# Эмпирические данные
plt.bar(k_values, obs_freq / num_batches, width=0.8,
        color='skyblue', edgecolor='black', alpha=0.7,
        label='Эмпирическая доля пакетов')

# Теоретическая модель
plt.plot(k_values, theoretical_probs, 'ro-', linewidth=2, markersize=6,
        label=f'Теоретическая Binomial PMF\n(n={n_batch}, p={p_hat:.3f})')

plt.xlabel('Количество вредоносных запросов в пакете (k)')
plt.ylabel('Относительная частота')
```

```
plt.legend()
plt.grid(True, alpha=0.3, axis='y')
```



Статистическая проверка гипотезы.

Используем критерий Хи-квадрат с группировкой "хвостов".

```
In [60]: # Функция для группировки хвостов
def group_tails(obs, exp, min_exp=5):
    obs_grouped = list(obs)
    exp_grouped = list(exp)

    # Группировка левого хвоста
    while len(exp_grouped) > 1 and exp_grouped[0] < min_exp:
        exp_grouped[1] += exp_grouped[0]
        obs_grouped[1] += obs_grouped[0]
        exp_grouped.pop(0)
        obs_grouped.pop(0)

    # Группировка правого хвоста
    while len(exp_grouped) > 1 and exp_grouped[-1] < min_exp:
        exp_grouped[-2] += exp_grouped[-1]
        obs_grouped[-2] += obs_grouped[-1]
        exp_grouped.pop()
        obs_grouped.pop()

    return np.array(obs_grouped), np.array(exp_grouped)

obs_grouped, exp_grouped = group_tails(obs_freq, expected_freq, min_exp=5)

print(f"Минимальная ожидаемая частота после группировки: {exp_grouped.min():.2f} (Дол")
print(f"Сумма наблюдаемых частот: {obs_grouped.sum()}")
print(f"Сумма ожидаемых частот: {exp_grouped.sum():.2f}")

# Применяем критерий Хи-квадрат
# ddof=1, так как мы оценили 1 параметр (p) из самих данных
chi2_stat, p_value = stats.chisquare(f_obs=obs_grouped, f_exp=exp_grouped, ddof=1)

print(f"\nКоличество категорий после группировки: {len(obs_grouped)}")
print(f"Статистика  $\chi^2$ : {chi2_stat:.4f}")
```

```
print(f"P-value: {p_value:.6f}")

alpha = 0.05
if p_value > alpha:
    print(f"\nВывод: P-value ({p_value:.6f}) > {alpha}. Мы НЕ ИМЕЕМ оснований отвергн
    print("Агрегированные данные статистически согласуются с биномиальным распределен
else:
    print(f"\nВывод: P-value ({p_value:.6f}) < {alpha}. Мы ОТВЕРГАЕМ H0.")
    print("Распределение значимо отличается от биномиального.")
```

Минимальная ожидаемая частота после группировки: 6.30 (Должно быть ≥ 5)

Сумма наблюдаемых частот: 100

Сумма ожидаемых частот: 100.00

Количество категорий после группировки: 10

Статистика χ^2 : 7.4769

P-value: 0.486153

Вывод: P-value (0.486153) > 0.05. Мы НЕ ИМЕЕМ оснований отвергнуть H_0 .

Агрегированные данные статистически согласуются с биномиальным распределением.

3. Качественные признаки

In [61]: `df.head(2)`

Out[61]:

	user_role	request_method	requests_per_min	payload_size_bytes	response_time_ms	cpu_lc
0	user	PUT	18	36.412739	98.430919	
1	user	GET	10	43.521982	7.556738	



Качественные (категориальные) признаки описывают не количество или размер, а принадлежность объекта к определённой категории (классу). В нашем датасете это:

- `user_role` (роль пользователя) — номинальный;
- `request_method` (метод HTTP-запроса) — номинальный;
- `severity_level` (уровень критичности события) — порядковый.

Особенности работы с качественными признаками.

1. Отсутствие математического смысла у арифметических операций.
2. Подготовка для машинного обучения: алгоритмы работают только с числами - качественные признаки требуют преобразования (кодирования).
3. Роль в кибербезопасности: резкое изменение распределения качественных признаков — один из самых надежных индикаторов атаки. Например, если в логах внезапно 95% запросов становятся методом `POST` (при норме 20%), это классический признак брутфорса.

3.1 Преобразование качественных признаков в категориальный тип

Для всех качественных признаков `pandas` определил тип данных `object`.

```
In [62]: df.dtypes
```

Out[62]:

user_role	object
request_method	object
requests_per_min	int64
payload_size_bytes	float64
response_time_ms	float64
cpu_load_percent	float64
source_port	int64
severity_level	object
is_malicious	int64

dtype: object

Для возможности эффективной работы с качественными признаками целесообразно преобразовать их в категориальный формат `category` .

Формат `category` — это специальный тип данных в `pandas` , который хранит значения как целочисленные коды с отдельным словарём категорий. Это даёт следующие преимущества:

- экономия памяти (в 5-10 раз);
- ускорение операций (`groupby` , `value_counts`);
- защита от опечаток (новые значения, которых нет в списке категорий, станут `NaN` — это полезно для очистки логов);
- корректный порядок категорий;
- корректные операции сравнения (`<` , `>`);
- корректные числовые характеристики.

Рекомендации по использованию формата `category` .

Ситуация	Рекомендация
Признак имеет небольшое количество уникальных значений	✅ Обязательно (экономия памяти)
Признак порядковый	✅ Обязательно
Признак номинальный с фиксированным набором	✅ Обязательно (валидация)
Признак имеет миллионы уникальных значений (ID, хэши)	❌ Не нужно (словарь будет огромным)
Признак текстовый с высокой вариативностью	❌ Не нужно

Рассмотрим преобразование номинальных признаков на примере признака `user_role` .

Признак имеет небольшое количество категорий, поэтому его целесообразно преобразовать в категориальный тип.

```
In [63]: df['user_role'].unique()
```

```
Out[63]: array(['user', 'guest', 'admin'], dtype=object)
```

Самый простой способ преобразования - использовать метод `astype('category')`.

```
In [64]: df1 = df.copy()
df1['user_role_cat'] = df1['user_role'].astype('category')

print(f"Тип: {df1['user_role_cat'].dtype}")
```

Тип: category

У признака в формате `category` появляется специальный доступ через `.cat`, позволяющий управлять категориями (добавлять, удалять, менять порядок).

Чтобы вывести перечень категорий признака `category`, используется свойство `cat.categories`.

При этом необходимо учитывать, что категории возвращаются в отсортированном лексикографическом порядке.

```
In [65]: print(df1['user_role_cat'].cat.categories)
```

Index(['admin', 'guest', 'user'], dtype='object')

Свойство `cat.ordered` показывает, имеют ли категории упорядоченную связь (`True`) или нет (`False`).

```
In [66]: print(f"Упорядочен: {df1['user_role_cat'].cat.ordered}")
```

Упорядочен: False

Оценим экономию памяти.

```
In [67]: mem_before = df1['user_role'].memory_usage(deep=True)
mem_after = df1['user_role_cat'].memory_usage(deep=True)
print(f"\nПамять до преобразования: {mem_before / 1024:.2f} KB")
print(f"Память после преобразования: {mem_after / 1024:.2f} KB")
print(f"Экономия: {(1 - mem_after/mem_before)*100:.1f}%")
```

Память до преобразования: 260.16 KB

Память после преобразования: 5.27 KB

Экономия: 98.0%

Создание неупорядоченных категорий для номинальных признаков корректно блокирует выполнение операций сравнения.

Будет выдана ошибка "TypeError: неупорядоченные категориальные переменные могут сравниваться только на равенство или неравенство".

```
In [68]: try:
        result = df1['user_role_cat'] > 'guest'
        print("❌ ОШИБКА: сравнение сработало")
    except TypeError as e:
```

```
print(f"\nКорректно: сравнение заблокировано ({type(e).__name__})")
print("guest < user — бессмысленное утверждение для номинального признака")
```

Корректно: сравнение заблокировано (TypeError)

guest < user — бессмысленное утверждение для номинального признака

Рассмотрим преобразование порядковых признаков на примере признака

`severity_level`.

У признака `severity_level` есть осмысленный порядок категорий: `Low < Medium < High < Critical`. Поэтому при его преобразовании в тип `category` нужно явно его задать.

Для этого рекомендуется использовать специализированный тип данных

`pandas.CategoricalDtype`, который позволяет явно задать характеристики категориальных переменных

- `categories` — последовательность уникальных значений, которые могут встречаться в данных. Они не должны содержать пропусков (`NaN`);
- `ordered` — булево значение, определяющее, считаются ли категории упорядоченными. По умолчанию `False` (неупорядоченные). Значение `None` позволяет сохранить порядок из уже существующих категориальных данных.

```
In [69]: severity_order = pd.CategoricalDtype(categories=['Low', 'Medium', 'High', 'Critical']
                                              ordered=True)
df1['severity_cat'] = df1['severity_level'].astype(severity_order)

print(f"Тип: {df1['severity_cat'].dtype}")
print(f"Категории: {df1['severity_cat'].cat.categories.tolist()}")
print(f"Упорядочен: {df1['severity_cat'].cat.ordered}")
```

Тип: category

Категории: ['Low', 'Medium', 'High', 'Critical']

Упорядочен: True

После такого преобразования разблокируются операции сравнения и они выполняются корректно.

```
In [70]: df1['severity_cat'].head()
```

```
Out[70]:
```

	severity_cat
0	Medium
1	Low
2	Low
3	Low
4	Low

dtype: category

```
In [71]: high_or_above = df1[df1['severity_cat'] >= 'High']
compare = df1.loc[0, 'severity_cat'] > df1.loc[1, 'severity_cat']
print(f"\nСравнение работает: событий >= High: {len(high_or_above)}")
print(f"\nСравнение работает: значение Medium > Low: {compare}")
```

Сравнение работает: событий $\geq$ High: 1124

Сравнение работает: значение Medium > Low: True

Демонстрация опасности неправильного преобразования порядковых признаков.

Преобразуем признак `severity` как обычный `category` (без порядка).

```
In [72]: df1['severity_wrong'] = df1['severity_level'].astype('category')

print("Категории (случайный порядок!):", df1['severity_wrong'].cat.categories.tolist())

try:
    result = df1['severity_wrong'] > 'Medium'
    print("Сравнение сработало (не должно было!)")
except TypeError as e:
    print(f"Корректно: сравнение заблокировано для неупорядоченных категорий")
```

Категории (случайный порядок!): ['Critical', 'High', 'Low', 'Medium']

Корректно: сравнение заблокировано для неупорядоченных категорий

Пример практического применения в кибербезопасности — фильтрация событий.

```
In [73]: # С ordered category фильтрация становится интуитивной и безопасной
critical_events = df1[df1['severity_cat'] == 'Critical']
high_plus = df1[df1['severity_cat'] >= 'High']
medium_to_high = df1[(df1['severity_cat'] >= 'Medium') & (df1['severity_cat'] <= 'High')]

print(f"Critical: {len(critical_events)}")
print(f"High + Critical: {len(high_plus)}")
print(f"Medium + High: {len(medium_to_high)}")
```

Critical: 382

High + Critical: 1124

Medium + High: 2178

3.2 Числовые характеристики качественных признаков

Набор числовых характеристик для номинальных и порядковых признаков различается.

Числовые характеристики категориальных признаков.

Характеристика	Номинальный	Порядковый	Формула / Определение	Смысл в кибербезопасности
Мода (Mo)	✓	✓	Категория с максимальной частотой: $Mo = \arg \max(P(x))$	Наиболее типичное состояние системы (базовый профиль).
Индекс вариации (VR)	✓	✓	$VR = 1 - \frac{f_{mode}}{N}$	Доля событий, не являющихся модальными. Мера предсказуемости.
Энтропия Шеннона (H)	✓	✓	$H = - \sum_{i=1}^K p_i \log_2(p_i)$	Мера хаоса/неопределённости. Максимальна при

Характеристика	Номинальный	Порядковый	Формула / Определение	Смысл в кибербезопасности
				равномерном распределении.
Медиана (Me)	✗	✓	Категория, для которой кумулятивная частота впервые ≥ 0.5	«Серединный» уровень угрозы. 50 % событий менее серьёзные, 50 % — более.
Квантили (Q_1, Q_3)	✗	✓	Категории, где кумулятивная частота ≥ 0.25 и ≥ 0.75	Границы «нормального» разброса критичности (межквартильный размах).
Математическое ожидание	✗	✗	Не применимо	Расстояния между категориями (Low → Med vs High → Critical) не равны.

Рассмотрим технологии расчета числовых характеристик номинальных признаков на примере признака `user_role`.

```
In [74]: # Частоты и доли (абсолютные и относительные)
counts = df['user_role'].value_counts() # сколько раз встречается каждая роль
probs = df['user_role'].value_counts(normalize=True) # доля каждой роли

# Самая частая роль (мода) и её доля
mode_role = counts.idxmax()
mode_share = probs.max()

print(f'Абсолютная частота категорий: \n{counts}')
print(f'\nОтносительная частота категорий: \n{probs}')
print(f'\nСамая частая роль (мода) = {mode_role} (доля {mode_share*100}%)')
```

Абсолютная частота категорий:

```
user_role
user      3724
guest     1017
admin       259
Name: count, dtype: int64
```

Относительная частота категорий:

```
user_role
user      0.7448
guest     0.2034
admin     0.0518
Name: proportion, dtype: float64
```

Самая частая роль (мода) = user (доля 74.48%)

Как можно использовать.

В норме у сотрудника или сервиса есть типичная роль (например, `user`). Если тот же аккаунт внезапно действует под редкой ролью (например, `backup_admin`), это может быть компрометация или эскалация привилегий.

Энтропия показывает, насколько «размазано» распределение ролей. Низкая энтропия = доминирует 1–2 роли; высокая = много разных ролей примерно поровну.

```
In [75]: # ЭНТРОПИЯ ШЕННОНА (в битах)

# список категорий
categories = df['user_role'].unique().tolist()

entropy = stats.entropy(probs, base=2)
max_entropy = np.log2(len(categories)) # Максимум для 3 категорий
print(f"ЭНТРОПИЯ ШЕННОНА (H): {entropy:.4f} бит (макс. возможная: {max_entropy:.4f} б
print(f"(Коэффициент заполнения: {(entropy/max_entropy)*100:.1f}%)")
```

ЭНТРОПИЯ ШЕННОНА (H): 1.0052 бит (макс. возможная: 1.5850 бит)
(Коэффициент заполнения: 63.4%)

Как можно использовать.

- Если у пользователя в сессии (набор событий) роли почти всегда одинаковые — энтропия низкая (норма).
- Если в одной сессии внезапно «перемешались» очень разные роли — энтропия высокая (подозрительно).

Рассмотрим технологии расчета числовых характеристик порядковых признаков на примере признака `severity_level`.

Для порядковых категориальных признаков существует строгий математический аппарат, который позволяет количественно оценить неопределенность, разброс и центральную тенденцию категориальных данных.

Например, для признака `severity_level` это особенно критично, так как позволяет перевести качественные оценки аналитиков SOC в строгие метрики для систем мониторинга.

Создадим исходные данные для расчета числовых характеристик.

```
In [76]: # количество наблюдений
n_samples = 5000

# абсолютные частоты встречаемости каждой категории
# value_counts(sort=False) сохраняет порядок категорий, заданный в dtype
counts = df1['severity_cat'].value_counts(sort = False)

# относительные частоты встречаемости каждой категории
probs = df1['severity_cat'].value_counts(normalize=True)

# накопленные относительные частоты
cum_probs = np.cumsum(probs)

# список категорий
categories = df1['severity_cat'].cat.categories.tolist()

print(f'Категории {categories}')
print(f'\nАбсолютные частоты встречаемости каждой категории {counts}')
print(f'\nОтносительные частоты встречаемости каждой категории {probs}')
print(f'\nНакопленные относительные частоты {cum_probs}')
```

Категории ['Low', 'Medium', 'High', 'Critical']

Абсолютные частоты встречаемости каждой категории severity_cat

Low	2440
Medium	1436
High	742
Critical	382

Name: count, dtype: int64

Относительные частоты встречаемости каждой категории severity_cat

Low	0.4880
Medium	0.2872
High	0.1484
Critical	0.0764

Name: proportion, dtype: float64

Накопленные относительные частоты severity_cat

Low	0.4880
Medium	0.7752
High	0.9236
Critical	1.0000

Name: proportion, dtype: float64

```
In [77]: # МОДА
mode_severity = counts.idxmax()
mode_share = probs.max()

print(f'МОДА (Mo)= {mode_severity} (доля {mode_share*100}%)')
```

МОДА (Mo)= Low (доля 48.8%)

```
In [78]: # МЕДИАНА
# Находим индекс, для которого кумулятивная относительная частота
# впервые >= 0.5
median_idx = np.searchsorted(cum_probs, 0.50)

# КВАНТИЛИ
q1_idx = np.searchsorted(cum_probs, 0.25)
q3_idx = np.searchsorted(cum_probs, 0.75)

print(f"МЕДИАНА (Me, 50-й перцентиль): '{categories[median_idx]}'")
print(f"1-й квартиль (Q1, 25-й перцентиль): '{categories[q1_idx]}'")
print(f"3-й квартиль (Q3, 75-й перцентиль): '{categories[q3_idx]}'")
```

МЕДИАНА (Me, 50-й перцентиль): 'Medium'
1-й квартиль (Q1, 25-й перцентиль): 'Low'
3-й квартиль (Q3, 75-й перцентиль): 'Medium'

```
In [83]: # ИНДЕКС ВАРИАЦИИ (Variation Ratio)
vr = 1 - (counts[mode_severity] / n_samples)
print(f"ИНДЕКС ВАРИАЦИИ (VR): {vr:.4f}")
print(f"(Доля событий, НЕ относящихся к моде. Чем ближе к 1, тем выше разнообразие)")
```

ИНДЕКС ВАРИАЦИИ (VR): 0.5120
(Доля событий, НЕ относящихся к моде. Чем ближе к 1, тем выше разнообразие)

```
In [84]: # ЭНТРОПИЯ ШЕННОНА (в битах)
entropy = stats.entropy(probs, base=2)
max_entropy = np.log2(len(categories)) # Максимум для 4 категорий = 2.0 бита
print(f"ЭНТРОПИЯ ШЕННОНА (H): {entropy:.4f} бит (макс. возможная: {max_entropy:.4f} бита)")
print(f"(Коэффициент заполнения: {(entropy/max_entropy)*100:.1f}%)")
```

ЭНТРОПИЯ ШЕННОНА (H): 1.7140 бит (макс. возможная: 2.0000 бит)
(Коэффициент заполнения: 85.7%)

Интерпретация в кибербезопасности.

Вычисленные метрики не просто описывают данные, они являются триггерами для систем обнаружения аномалий (UEBA/SIEM). Рассмотрим, как меняются эти характеристики при переходе от нормального состояния к атаке.

Сценарий: начало масштабной атаки (например, брутфорс или DDoS). Представьте, что злоумышленник начал генерировать тысячи запросов, которые классификатор помечает как **High** или **Critical**.

Метрика: мода (Mo). Нормальное состояние (Baseline): **Low** или **Medium**. Состояние под атакой: резко меняется на **High** или **Critical**. Действие системы безопасности: сигнал "смена базового профиля системы".

Метрика: индекс вариации (VR). Нормальное состояние (Baseline): высокий (~0.45–0.60). События разнообразны. Состояние под атакой: резко падает (например, до 0.10). Почти все события стали одинаковыми (**High**). Действие системы безопасности: сигнал "падение VR = признак автоматизированной, однообразной активности (бот)".

Метрика: энтропия (H). Нормальное состояние (Baseline): средняя/высокая (~1.3–1.6 бит). Состояние под атакой: резко падает (стремится к 0). Неопределенность исчезает, мы точно знаем, что будет значение **High**. Действие системы безопасности: сигнал "падение энтропии категориального признака — классический индикатор сканирования или флуда".

Метрика: медиана (Me). Нормальное состояние (Baseline): **Low**. Состояние под атакой: сдвигается вправо: **Medium** → **High**. Действие системы безопасности: сигнал "эскалация: если скользящая медиана за 15 минут превышает **Medium**, событию автоматически присваивается наивысший приорит".

Метрика: межквартильный размах. Нормальное состояние (Baseline): от **Low** до **Medium**. Состояние под атакой: сдвигается вправо: сужается до одной категории (например, только **High**). Действие системы безопасности: сигнал "указывает на то, что атака носит массовый, а не точечный характер".

3.3 Описание распределения качественных признаков

Описание распределения качественных (категориальных) признаков — это фундамент построения базовых профилей (baseline) в системах безопасности. В отличие от количественных данных, где для анализа распределений используются эмпирические функции вероятностей, распределения, плотности вероятностей и их визуализации, для качественных данных мы оперируем частотами категорий.

Распределение качественных признаков описывается двумя способами:

1. Табличный (ряд распределения): таблица абсолютных (n) и относительных ($w = n/N$) частот для каждой категории.
2. Графический: столбчатая диаграмма (Bar Chart).

Важно: для порядковых признаков категории всегда располагаются в порядке возрастания, а не по частоте.

Для описания распределения качественных признаков используются три основные величины.

Пусть n — общий объем выборки.

Абсолютная частота n_i : сколько раз встретилась категория k_i :

$$n_i = \text{count}(x = k_i).$$

Относительная частота w_i : доля (вероятность) появления категории k_i .

Сумма относительных частот равна 1:

$$w_i = \frac{n_i}{n}.$$

Накопленная (кумулятивная) частота c_i : доля наблюдений, принадлежащих категории k_i или любой категории, стоящей перед ней:

$$c_i = \sum_{i=1}^k w_i.$$

При анализе распределения качественных признаков нужно учитывать различия между номинальными и порядковыми признаками.

Характеристика описания	Номинальный признак (напр., <code>request_method</code>)	Порядковый признак (напр., <code>severity_level</code>)
Смысл порядка категорий	Отсутствует. <code>GET</code> не больше и не меньше <code>POST</code> .	Присутствует. <code>Low < Medium < High < Critical</code>
Правило сортировки в таблице/графике	По убыванию частоты (для удобства чтения: самые частые слева)	Строго по логическому порядку (слева направо: от наименьшего к наибольшему)
Применимость накопленной частоты (S_i)	❌ Бессмысленна. Складывать доли <code>GET</code> и <code>POST</code> математически некорректно	✅ Критически важна. Показывает, какая доля событий имеет уровень X или ниже
Возможность нахождения числовых характеристик	❌ Невозможно	✅ Возможно

Рассмотрим технологию описания распределения номинальных признаков на примере признака `request_methods`.

Табличное описание распределения признака (ряд распределения).

```
In [85]: # Вычисляем частоты. sort=True сортирует по убыванию частоты (идеально для номинальны
nom_counts = df1['request_method'].value_counts(sort=True)
nom_probs = df1['request_method'].value_counts(sort=True, normalize = True)

print(f"{'Категория':<10} | {'Абс. частота (n)':<15} | {'Отн. частота (w)':<15}")
print("-" * 45)
```

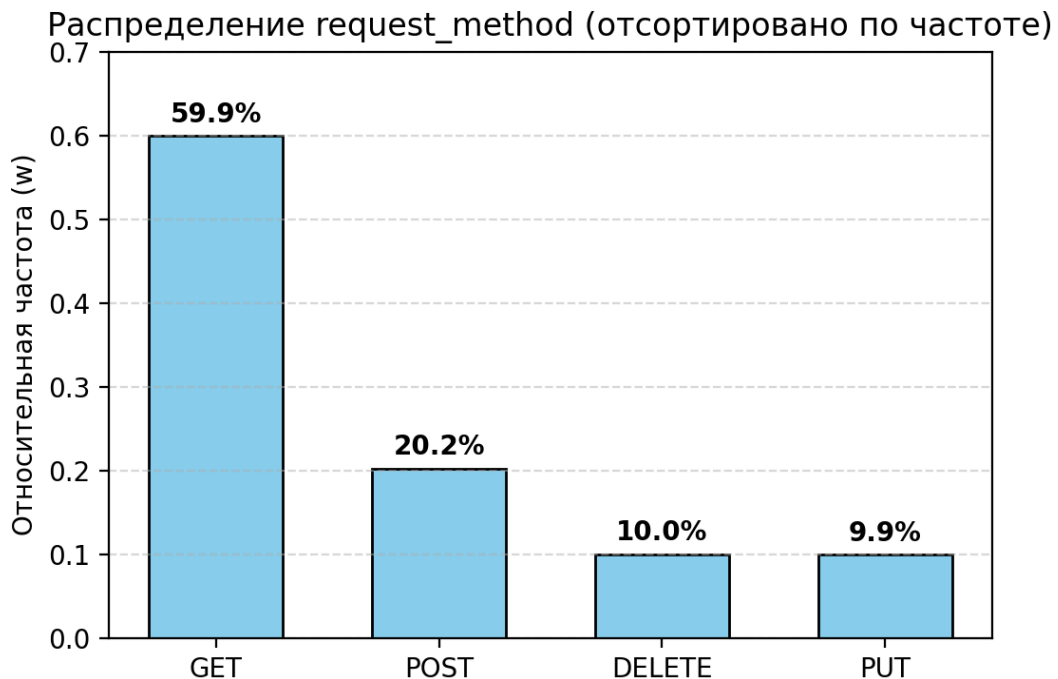
```
for cat, count, prob in zip(nom_counts.index, nom_counts.values, nom_probs.values):
    print(f"{cat:<10} | {count:<15} | {prob:.2%}")
```

Категория	Абс. частота (n)	Отн. частота (w)
GET	2995	59.90%
POST	1010	20.20%
DELETE	499	9.98%
PUT	496	9.92%

Визуальное описание распределения признака (столбчатая диаграмма).

```
In [86]: plt.figure(figsize=(6, 4))
bars = plt.bar(nom_counts.index, nom_probs.values,
               color='skyblue', edgecolor='black', width=0.6)
for bar, prob in zip(bars, nom_probs.values):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f'{prob:.1%}', ha='center', va='bottom', fontweight='bold')

plt.title('Распределение request_method (отсортировано по частоте)')
plt.ylabel('Относительная частота (w)')
plt.ylim(0, 0.7)
plt.grid(axis='y', linestyle='--', alpha=0.5)
```



Интерпретация результатов в кибербезопасности.

Базовая линия: запросы GET составляют 60%, а POST — 25%. Это нормальный профиль веб-приложения.

Детекция аномалии: если в реальном времени мы строим такое же распределение за последние 10 минут и видим, что POST вырос до 85%, а GET упал до 10%, это индикатор атаки. Скорее всего, идет брутфорс формы входа, спам-атака или работа веб-шелла. Номинальные признаки детектируют атаки через резкую смену моды.

Рассмотрим технологию описания распределения порядковых признаков на примере признака `severity_level`.

Табличное описание распределения признака (ряд распределения) и эмпирическая функция распределения (CDF).

```
In [88]: # Вычисляем частоты
ord_counts = df1['severity_cat'].value_counts(sort=False)
ord_probs = df1['severity_cat'].value_counts(sort=False, normalize = True)
ord_cum_probs = np.cumsum(ord_probs)

# Создаём DataFrame
freq_table = pd.DataFrame({'Категория': ord_counts.index,
                           'Абс. частота (n_i)': ord_counts.values,
                           'Отн. частота (w_i)': ord_probs.values,
                           'Накопленная частота (c_i)': ord_cum_probs})

freq_table['Отн. частота (w_i)'] = freq_table['Отн. частота (w_i)'].apply(lambda x: f
freq_table['Накопленная частота (c_i)'] = freq_table['Накопленная частота (c_i)'].app
freq_table
```

Out[88]:

	Категория	Абс. частота (n_i)	Отн. частота (w_i)	Накопленная частота (c_i)
severity_cat				
Low	Low	2440	48.80%	48.80%
Medium	Medium	1436	28.72%	77.52%
High	High	742	14.84%	92.36%
Critical	Critical	382	7.64%	100.00%

Замечание.

По строгому математическому определению функции распределения $F = P(X < x)$ накопленная частота до значения **Low** должна равняться 0, потому что нет категорий «меньше **Low**».

Однако в практике анализа порядковых категориальных данных и особенно в задачах кибербезопасности, где важны «доля событий не выше уровня X», чаще используют нестрогую версию: $F = P(X \leq x)$. Тогда CDF в точке **Low** равна доле всех **Low**, в точке **Medium** — доле **Low + Medium** и т.д.

Визуальное описание распределения признака (столбчатая диаграмма + кумулятивная функция распределения (CDF)).

```
In [89]: fig, ax1 = plt.subplots(figsize=(6, 4))

# График 1: Относительные частоты (столбцы)
colors = ['#90EE90', '#FFD700', '#FFA500', '#FF4500'] # Цвета по возрастанию серьезно
bars = ax1.bar(ord_counts.index, ord_probs.values, color=colors,
               edgcolor='black', width=0.6, alpha=0.8)
ax1.set_ylabel('Относительная частота ($w_i$)', color='black')
ax1.set_ylim(0, 0.65)
ax1.set_title('Распределение severity_level (упорядочено по логике, а не по частоте)')

# График 2: Накопленная частота (ступенчатая линия CDF)
ax2 = ax1.twinx()
ax2.step(range(len(categories)), cum_probs, where='post',
```

```

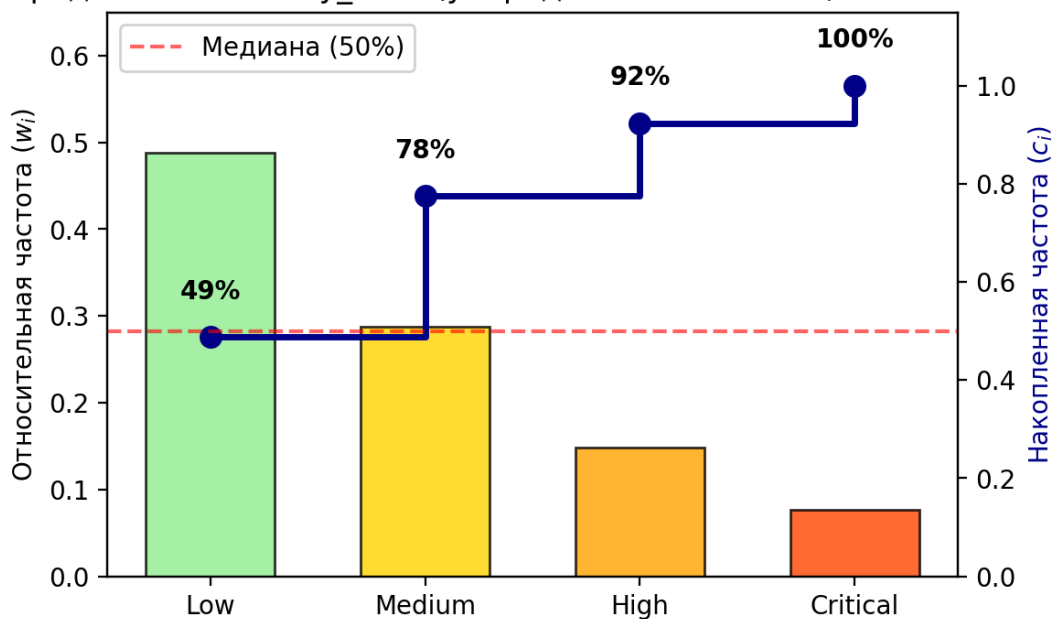
        color='darkblue', linewidth=2.5, marker='o', markersize=8)
ax2.set_ylabel('Накопленная частота ($c_i$)', color='darkblue')
ax2.set_ylim(0, 1.15)
ax2.set_xticks(range(len(ord_counts)))
ax2.set_xticklabels(ord_counts.index)

# Аннотации для чтения квантилей с графика CDF
for i, (cat, cum) in enumerate(zip(ord_counts.index, ord_cum_probs)):
    ax2.annotate(f'{cum:.0%}', (i, cum), textcoords="offset points",
                xytext=(0, 15), ha='center', fontweight='bold')

# Линия медианы (50%)
ax2.axhline(y=0.5, color='red', linestyle='--', alpha=0.6, label='Медиана (50%)')
ax2.legend(loc='upper left');

```

Распределение severity_level (упорядочено по логике, а не по частоте)



Интерпретация результатов в кибербезопасности.

Ряд распределения: строка **Medium** показывает: относительная частота $w \approx 30.00\%$, а накопленная частота - $c_i = 78.00\%$. Это означает, что 78% всех событий имеют уровень **Medium** или ниже (**Low** + **Medium**).

График (CDF): ступенчатая синяя линия пересекает красную пунктирную линию 50% (медиану) на уровне категории **Low**. Это значит, что медианная критичность событий в системе равна **Low**.

Детекция аномалии: предположим, что началась DDoS-атака или массовое сканирование уязвимостей. Генератор событий начнет массово присваивать уровень **High** или **Critical**. На графике это проявится как резкое изменение формы ступенчатой линии: она станет более полой в начале и резко взлетит в конце. Если накопленная частота для **Low** + **Medium** упадет с 85% до 40%, система должна автоматически повысить уровень тревоги, так как "центр тяжести" инцидентов сместился в опасную зону.

Литература

1. Титов А.Н. Решение задач теории вероятностей и математической статистики в Python : учеб.-метод. пособие / Р.Ф. Тазиева; Казан. нац. исслед. технол. ун-т; А.Н. Титов. — Казань : КНИТУ, 2022. — 144 с.
2. Груздев А. В., Хейдт М. Изучаем Pandas. 2-ое изд., испр. и доп. - Москва: ДМК Пресс, 2019. - 1700 с.
3. Протоdjяконов А.В. Алгоритмы Data Science и их практическая реализация на Python : учебное пособие / А.В. Протоdjяконов, П.А. Пылов, В.Е. Садовников. - Москва : Инфра-Инженерия, 2022. - 392 с.
4. Демидова, Л. А. Разведочный анализ данных. Python : учебно-методическое пособие / Л. А. Демидова. — Москва : РТУ МИРЭА, 2022 — Часть 1 — 2022. — 107 с.
5. Демидова, Л. А. Разведочный анализ данных. Python : учебно-методическое пособие / Л. А. Демидова. — Москва : РТУ МИРЭА, 2023 — Часть 2 — 2023. — 92 с.